

PYTHON EXCEPTION HANDLING AND ERROR MANAGEMENT THE COMPLETE PROFESSIONAL HANDBOOK

From Beginner Fundamentals to Production-Grade Error Management

Tracebacks, try/except/else/finally, Custom Exceptions, Chaining, Logging, Retry Logic, Data Science, Machine Learning, APIs, Real-World Projects and Interview Preparation

Kajola Gbenga
Data Scientist & AI Engineer

Table of Contents

1. Introduction to Errors and Exceptions
2. Understanding Python Tracebacks
3. Common Python Exceptions (Built-in Reference)
4. The try Statement
5. except
6. else
7. finally
8. Complete Exception Handling Flow
9. Raising Exceptions with raise
10. Custom Exceptions
11. Exception Hierarchy
12. Exception Objects
13. Exception Chaining
14. Re-Raising Exceptions
15. Assertions
16. Exception Handling with Functions
17. Exception Propagation
18. Nested Exception Handling
19. File Handling Exceptions
20. Exceptions in Data Science
21. Exceptions in Machine Learning and AI
22. Exceptions in APIs and Web Applications
23. Exceptions and Context Managers
24. Exceptions and Classes/OOP
25. Logging and Exceptions
26. Good vs Bad Exception Handling
27. Exception Handling Best Practices
28. Debugging Exception-Related Problems
29. Exception Handling Design Patterns
30. Retry Logic
31. Real-World Projects
32. Interview Questions
33. Practical Exercises
34. Complete Exception Handling Cheat Sheet

1. Introduction to Errors and Exceptions

1.1 What Is an Error?

An error, in the broadest sense, is anything that prevents a program from behaving the way it was intended to. Python programmers use the word 'error' loosely to cover several genuinely different situations: code that Python cannot even parse (a syntax error), code that runs but fails partway through (a runtime error/exception), and code that runs to completion without crashing but produces the wrong result anyway (a logical error). Distinguishing these three carefully, which this section does, is the essential first step to becoming effective at diagnosing and fixing problems in real code.

1.2 What Is an Exception?

An exception is a specific, well-defined kind of runtime error: an event that occurs during program execution which disrupts the normal flow of instructions. When Python encounters a situation it cannot proceed from normally, dividing by zero, accessing a list index that does not exist, calling a method on an object that does not support it, it does not simply stop silently; it raises an exception, an object that carries information about what went wrong, where, and why. If nothing in the program catches and handles that exception, the program terminates and Python prints a traceback (covered fully in Part 2).

Critically, an exception is not automatically a flaw in the program's logic. Exceptions are a normal, expected part of programming: a file genuinely might not exist, a network request genuinely might time out, a user genuinely might type invalid input. Exception handling, the subject of this entire handbook, is the discipline of anticipating these situations and responding to them deliberately rather than letting the program crash.

1.3 Errors vs Exceptions: The Precise Distinction

Code Line	Explanation
Error (general term)	An umbrella term covering anything that goes wrong: syntax problems, runtime exceptions, and logical mistakes alike. Not a precise Python technical term on its own.
Exception (precise term)	A specific Python object, an instance of a class deriving from <code>BaseException</code> , raised at runtime to signal a disruption; it is Python's formal mechanism for representing and communicating runtime errors, and it can be caught and handled with <code>try/except</code> .
<code>SyntaxError</code> (a special case)	Technically implemented as an Exception subclass in Python, but practically distinct: it is detected before the program ever starts running, during parsing, and cannot be caught by wrapping the offending code in <code>try/except</code> , because the code never successfully compiles into runnable instructions in the first place.

1.4 Syntax Errors

A syntax error occurs when code violates Python's grammar rules and cannot be parsed into valid instructions at all. Python detects these before execution begins, while translating the source code into bytecode.

```
def greet(name)
    print(f'Hello, {name}')
```

Code Line	Explanation
The problem	The function definition is missing its trailing colon after the parameter list.
The output	SyntaxError: expected ':'
Why it cannot be caught with try/except	Python must fully parse a block of code into bytecode before any of it can run; if the parser itself fails, execution of that code never begins, so there is no running program yet in which a try block could intervene.

1.5 Runtime Errors (Exceptions)

A runtime error occurs after the code has successfully parsed and started running, when an operation cannot complete as written. This is what Python formally calls an exception.

```
numbers = [10, 20, 30]
print(numbers[5])
```

Code Line	Explanation
The problem	The list numbers only has valid indices 0, 1, and 2; index 5 does not exist.
The output	IndexError: list index out of range
Why this is a true exception	The code is syntactically valid and begins executing normally; the failure only happens at the specific moment Python tries to actually access index 5, which is precisely the kind of situation try/except is designed to catch.

1.6 Logical Errors

A logical error is the most dangerous and subtle category: the code is syntactically correct and runs to completion without raising any exception at all, but it produces an incorrect result because the underlying logic is flawed. Python has no way to detect these automatically, since nothing about them looks wrong from the interpreter's point of view.

```
def calculate_average(numbers):
    total = sum(numbers)
    return total / len(numbers) - 1 # bug: subtracting 1 unintentionally

print(calculate_average([10, 20, 30]))
```

Code Line	Explanation
The problem	The function runs without error and returns 19.0 instead of the mathematically correct average, 20.0, because of a stray '- 1' that should not be there.
Why this is dangerous	No exception is raised, no traceback appears, and casual testing might not catch it; logical errors are typically found through careful testing, code review, or comparing output against known-correct expected values, not through exception handling.

1.7 Exceptions vs Bugs

A 'bug' is a broad, informal term for any defect in a program, whether that defect manifests as a syntax error, an unhandled exception, or a logical error. Every unhandled exception traces back to some bug (either in the code itself or in an unvalidated assumption about its input or environment), but not every bug produces an exception; a logical error is a bug that produces no exception at all. Exception handling is one important tool for managing certain kinds of bugs, specifically ones that manifest as a runtime failure, but it is not a substitute for testing, code review, and careful reasoning about logic, which are what catch the bugs that never raise an exception in the first place.

1.8 Why Exceptions Occur

- Invalid input from a user, file, API, or another part of the system that the code did not anticipate.
- Resources that are unavailable: a missing file, an unreachable network host, insufficient memory.
- Operations that are mathematically or logically undefined, like dividing by zero.
- Incorrect assumptions about data: assuming a key exists in a dictionary, assuming a list has at least one element, assuming a value has already been converted to the right type.
- External systems failing or behaving unexpectedly: a database connection dropping, an API returning malformed data, a third-party service timing out.
- Programmer mistakes: typos in variable names, calling a method that does not exist on a given object, off-by-one errors in indexing.

1.9 Why Exception Handling Is Important

- It lets a program respond to a problem deliberately (retry, use a fallback value, inform the user clearly) instead of crashing unpredictably.
- It separates the normal, successful path of a program's logic from the handling of abnormal situations, keeping both easier to read.
- It allows a program to clean up resources reliably (closing files, releasing database connections) even when something goes wrong partway through an operation.
- It is essential for building production software, where an unhandled crash can mean lost data, a failed transaction, or a broken user experience, none of which are acceptable in a real system.
- It provides rich diagnostic information (via the traceback and the exception object itself) that dramatically speeds up debugging compared to a program that merely produces a wrong answer silently.

1.10 What Happens When Python Encounters an Unhandled Exception

```
def divide(a, b):  
    return a / b  
  
print('Starting calculation')  
result = divide(10, 0)  
print('This line never runs')
```

When Python executes this code, 'Starting calculation' is printed normally, but the call to `divide(10, 0)` raises a `ZeroDivisionError` inside the function. Because nothing in this code catches that exception, Python immediately stops normal execution at that exact point: the final `print()` call never runs, the program terminates with a non-zero exit status (signaling failure to the operating system or calling process), and Python prints a full traceback to standard error, describing exactly what happened and where. Understanding and reading that traceback output is the entire subject of Part 2.

1.11 Errors, Exceptions, and Warnings

It is worth briefly distinguishing exceptions from warnings, since both can appear in program output. A warning (raised with Python's `warnings` module, for example `DeprecationWarning`) signals that something is questionable or discouraged, but does not stop execution; the program continues running normally after a warning is issued. An exception, by contrast, always disrupts the normal flow of execution unless it is explicitly caught. This handbook focuses entirely on exceptions, not warnings, since warnings are a distinct (and comparatively minor) mechanism.

2. Understanding Python Tracebacks

2.1 What a Traceback Is

A traceback (sometimes called a stack trace) is the block of text Python prints when an exception propagates all the way up through a program without being caught. It is not an error message on its own; it is a structured report describing the entire chain of function calls that were active at the moment the exception was raised, working from where execution started down to the exact line that triggered the failure. Learning to read a traceback fluently is one of the single highest-leverage skills in professional Python debugging.

2.2 Anatomy of a Traceback

```
def get_price(item):
    prices = {'apple': 1.50, 'banana': 0.75}
    return prices[item]

def checkout(cart):
    total = 0
    for item in cart:
        total += get_price(item)
    return total

checkout(['apple', 'mango'])
```

```
Traceback (most recent call last):
  File "store.py", line 11, in <module>
    checkout(['apple', 'mango'])
  File "store.py", line 8, in checkout
    total += get_price(item)
              ^^^^^^^^^^^^^
  File "store.py", line 3, in get_price
    return prices[item]
           ~~~~~^~~~~~
KeyError: 'mango'
```

Code Line	Explanation
Traceback (most recent call last):	A fixed header line that always appears first, explicitly telling you the list of calls below is ordered from oldest (outermost) call at the top to most recent (innermost, where the failure actually happened) call at the bottom.
File "store.py", line 11, in <module>	Identifies the file and exact line number of one 'frame' in the call stack; <module> means this particular line is at the top level of the script, not inside any function.
checkout(['apple', 'mango'])	The actual source code found at that line, reproduced so you do not need to open the file to see what was executed.
File "store.py", line 8, in checkout	The next frame down: execution had moved into the checkout function, specifically to line 8.
The ^^^ / ~~~ markers	Modern Python (3.11 and later) underlines the specific sub-expression

Code Line	Explanation
	within the line that was actually being evaluated when the error occurred, narrowing down the exact operation at fault, which is especially helpful on long or complex lines.
File "store.py", line 3, in get_price	The innermost frame: execution had gone one level deeper still, into get_price, at line 3, which is exactly where the failure ultimately occurred.
KeyError: 'mango'	The final line, always the most important one to read first in practice: the exception type (KeyError) and its associated message ('mango', the specific key that was missing).

2.3 Reading Tracebacks From Bottom to Top

Although a traceback is printed top to bottom in call order (oldest call first), the most effective way to read one in practice is to start at the very bottom and work upward. The bottom line always tells you the two most important facts immediately: exactly what kind of problem occurred (the exception type) and what the specific triggering detail was (the message). From there, moving upward, each frame shows you the exact chain of function calls that led to that failure, letting you trace backward from the specific failing line, through each intermediate call, to where the whole chain began.

Practical Tip: A common beginner mistake is to read a long traceback top to bottom and get lost or overwhelmed partway through. Professional developers almost always jump straight to the last line first, then only scan upward as far as needed to understand the surrounding context.

2.4 The Call Stack

The call stack is the underlying data structure that makes tracebacks possible: it is the list of function calls that are currently active (that is, have started but not yet returned) at any given moment during execution. Every time a function is called, a new frame is pushed onto the stack; every time a function returns normally, its frame is popped off. When an exception is raised and not immediately caught, Python does not discard the stack; instead it uses that exact sequence of still-active frames, from the moment of the raise all the way back to the program's entry point, to construct the traceback.

2.5 A Second Realistic Traceback Example

```
import json

def load_config(path):
    with open(path) as f:
        return json.load(f)

config = load_config('settings.json')
```

```
Traceback (most recent call last):
  File "app.py", line 7, in <module>
    config = load_config('settings.json')
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "app.py", line 4, in load_config
```

```

with open(path) as f:
    ~~~~~
FileNotFoundError: [Errno 2] No such file or directory: 'settings.json'

```

Diagnosis: the last line immediately identifies both the exception type (`FileNotFoundError`) and the exact missing path (`'settings.json'`). Reading upward, the failure occurred inside `load_config`, specifically at the `open(path)` call, which was itself triggered by the top-level call to `load_config('settings.json')`. The fix is either to ensure the file exists at that path before calling `load_config`, to pass the correct path, or to explicitly catch `FileNotFoundError` and handle a missing configuration file gracefully (see Part 19).

2.6 A Third Example: A Deeper Call Chain

```

Traceback (most recent call last):
  File "main.py", line 20, in <module>
    run_pipeline(raw_data)
  File "main.py", line 15, in run_pipeline
    cleaned = clean_data(data)
  File "main.py", line 9, in clean_data
    normalized = normalize(row)
  File "main.py", line 4, in normalize
    return (value - mean) / std
    ~~~~~^~~~~
ZeroDivisionError: float division by zero

```

Code Line	Explanation
Diagnosis, working bottom to top	<code>ZeroDivisionError</code> happened inside <code>normalize()</code> , specifically dividing by <code>std</code> , which must be 0. Moving upward: <code>normalize</code> was called from <code>clean_data</code> , which was called from <code>run_pipeline</code> , which was called from the top-level script.
Root cause reasoning	<code>std</code> being 0 typically means every value being normalized was identical (zero variance), which is a data quality issue upstream of <code>normalize()</code> itself; the real fix likely belongs in <code>clean_data</code> (validate <code>std</code> before calling <code>normalize</code>) rather than inside <code>normalize</code> (which cannot know why <code>std</code> is 0).
Debugging Insight: This example illustrates an important debugging principle: the line where an exception is raised is not always the line where the underlying problem originated. The traceback tells you exactly where the program broke; determining why the data got into that broken state often requires reasoning about the calling frames above it, covered fully in Part 28.	

3. Common Python Exceptions (Built-in Reference)

This part works through Python's most important built-in exceptions in organized families, since exceptions within the same family tend to share a common cause and prevention strategy. For each exception you will find: a definition, why it occurs, a failing example with its exact error output, a corrected example, common causes, and prevention strategies.

3.1 The Root of Everything: BaseException and Exception

Code Line	Explanation
BaseException	The absolute root of Python's entire exception hierarchy; every exception, without exception, inherits from it either directly or indirectly. It is deliberately almost never caught directly in application code (see Part 11).
Exception	A direct subclass of BaseException, and the parent class of essentially every exception that represents an ordinary application-level error. Nearly every exception discussed in this handbook, and nearly every exception you should ever catch in real code, is a subclass of Exception, not a direct subclass of BaseException.

The full reasoning behind why BaseException and Exception are split apart the way they are, and why a small number of exceptions (KeyboardInterrupt, SystemExit, GeneratorExit) deliberately sit outside the Exception branch, is covered in complete depth in Part 11.

3.2 The ArithmeticError Family

3.2.1 ZeroDivisionError

```
result = 10 / 0
```

Code Line	Explanation
Definition	Raised when a division or modulus operation is attempted with a divisor of zero.
Error output	ZeroDivisionError: division by zero
Corrected example	if denominator != 0: result = numerator / denominator else: result = None (or handle with try/except, see Part 5)
Common causes	An average computed over an empty collection; a rate or ratio calculated before confirming the denominator is non-zero; user-supplied divisor not validated.
Prevention	Validate the divisor before dividing, or wrap the operation in try/except ZeroDivisionError.

3.2.2 OverflowError

```
import math
result = math.exp(1000)
```

Code Line	Explanation
Definition	Raised when the result of an arithmetic operation is too large to be represented; note that Python's built-in int type has arbitrary precision

Code Line	Explanation
	and essentially never overflows, so this is mostly encountered with float-producing operations like <code>math.exp</code> or <code>math.factorial</code> results assigned to a float.
Error output	<code>OverflowError: math range error</code>
Prevention	Validate input magnitude before calling functions known to grow extremely quickly, or use a library that supports arbitrary-precision arithmetic (like Python's own <code>int</code> , or the <code>decimal</code> module) when the true mathematical result must be preserved exactly.

3.3 The LookupError Family

3.3.1 IndexError

```
fruits = ['apple', 'banana']
print(fruits[5])
```

Code Line	Explanation
Definition	Raised when a sequence (list, tuple, string) is accessed using an index that is out of the valid range.
Error output	<code>IndexError: list index out of range</code>
Corrected example	<code>if len(fruits) > 5: print(fruits[5]) else: print('Not enough items')</code>
Common causes	Off-by-one mistakes in loop bounds; assuming a collection always has a minimum length; accessing the result of a filter or search that might have found nothing.
Prevention	Check <code>len()</code> before indexing, use <code>.get()</code> -style safe access patterns where available, or catch <code>IndexError</code> explicitly around code that indexes into data of uncertain length.

3.3.2 KeyError

```
config = {'host': 'localhost'}
print(config['port'])
```

Code Line	Explanation
Definition	Raised when a dictionary is accessed with a key that does not exist.
Error output	<code>KeyError: 'port'</code>
Corrected example	<code>port = config.get('port', 8080) # provides a sensible default instead of raising</code>
Common causes	Assuming a configuration or JSON-derived dictionary always contains every expected key; typos in key names; accessing data that depends on optional user input.
Prevention	Use <code>dict.get(key, default)</code> when a missing key should fall back to a default; use <code>'key'</code> in <code>dict</code> to check first when a missing key should be handled differently; catch <code>KeyError</code> explicitly when a missing key represents a genuine error condition worth reporting.

3.4 TypeError and ValueError

3.4.1 TypeError

```
total = 'Total: ' + 42
```

Code Line	Explanation
Definition	Raised when an operation or function is applied to an object of an inappropriate type, for example combining a string and an integer with +, or calling a function with the wrong number or kind of arguments.
Error output	TypeError: can only concatenate str (not "int") to str
Corrected example	total = 'Total: ' + str(42)
Common causes	Forgetting to convert a number to a string before concatenation; calling a function with a positional argument where a keyword was expected, or vice versa; passing None somewhere a real value was assumed to exist.
Prevention	Validate and convert types explicitly at boundaries (user input, file data, API responses) rather than assuming they already match; use type hints and a static checker to catch many of these before runtime.

3.4.2 ValueError

```
age = int('twenty-five')
```

Code Line	Explanation
Definition	Raised when an operation or function receives an argument of the correct type but an inappropriate or unparseable value.
Error output	ValueError: invalid literal for int() with base 10: 'twenty-five'
Corrected example	try: age = int(user_input) except ValueError: print('Please enter a valid number')
Common causes	Converting unvalidated user input to a number; unpacking a sequence into the wrong number of variables (a, b = [1, 2, 3] also raises ValueError); passing an out-of-range or nonsensical value to a function that checks its arguments.
Prevention	Validate input format before conversion where feasible, and always wrap conversions of external input (user input, file contents, API responses) in try/except ValueError.

TypeError vs ValueError: TypeError and ValueError are frequently confused. As a rule of thumb: TypeError means 'this is fundamentally the wrong kind of thing' (a string where a number was required); ValueError means 'this is the right kind of thing, but an unacceptable specific value' (a string that looks like it should be a number, but is not a parseable one).

3.5 AttributeError and NameError

3.5.1 AttributeError

```
text = 'hello'
text.push('world')
```

Code Line	Explanation
Definition	Raised when an attribute or method is accessed on an object that does not have it.
Error output	AttributeError: 'str' object has no attribute 'push'
Common causes	Confusing methods available on one type (like list.append) with a similar but different type (str has no push or append at all, since strings are immutable); a variable holding None unexpectedly, then calling a method on it; a typo in a method name.
Prevention	Know (or check with dir() or documentation) which methods a type actually supports; guard against None with an explicit check before calling a method on a value that might not have been set.

3.5.2 NameError and UnboundLocalError

```
print(total)

def show_total():
    print(total)
    total = 100
```

Code Line	Explanation
NameError definition	Raised when a name (variable or function) is referenced before it has been assigned anywhere in an accessible scope. print(total) on its own, with no prior definition of total anywhere, raises: NameError: name 'total' is not defined.
UnboundLocalError definition	A more specific subclass of NameError raised when a local variable inside a function is referenced before being assigned within that same function, even though it might exist at module level; calling show_total() above raises: UnboundLocalError: cannot access local variable 'total' where it is not associated with a value, because Python sees the later assignment total = 100 and treats total as local to the whole function, making the earlier print(total) reference an as-yet-unassigned local.
Common causes	Typos in variable names; using a variable before the line that defines it; assigning to a variable anywhere inside a function without first declaring it global or nonlocal, which silently makes it local for the entire function body.
Prevention	Define variables before use; when a function genuinely needs to modify a variable from an enclosing scope, use the global or nonlocal keyword explicitly (covered in your Functions notes) rather than relying on implicit scoping.

3.6 The Import Family

3.6.1 ImportError and ModuleNotFoundError

```
import numpy
```

Code Line	Explanation
Definition	ModuleNotFoundError (a subclass of ImportError, introduced in Python 3.6) is raised when a module referenced in an import

Code Line	Explanation
	statement cannot be found at all. The broader <code>ImportError</code> is raised for related problems, such as successfully finding a module but failing to import a specific name from it.
Error output	<code>ModuleNotFoundError: No module named 'numpy'</code>
Common causes	A typo in the module name (as above, 'numpy' instead of 'numpy'); a package that was never installed in the current environment; running code in the wrong virtual environment; a genuinely optional dependency that a user has not installed.
Prevention	Double-check spelling; ensure the package is listed in the project's dependencies and installed (<code>pip install</code>); for genuinely optional dependencies, catch <code>ImportError</code> at the top of a module and provide a clear message or a fallback code path.

3.7 The File and OS Family

3.7.1 `FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `NotADirectoryError`

Code Line	Explanation
<code>FileNotFoundError</code>	Raised when an operation (like <code>open()</code>) references a file path that does not exist. Common cause: a typo in the path, a relative path resolved from an unexpected working directory, or a file that was expected to already exist but has not been created yet.
<code>PermissionError</code>	Raised when the operating system denies access to a resource, for example trying to write to a read-only file or a directory the current user lacks permission for.
<code>IsADirectoryError</code>	Raised when code tries to open a directory path as if it were a regular file for reading or writing.
<code>NotADirectoryError</code>	Raised when code tries to treat a regular file's path as if it were a directory, for example attempting to list its contents.
<code>OSError</code> (the parent of all of these)	The general base class for operating-system-related errors; <code>FileNotFoundError</code> , <code>PermissionError</code> , <code>IsADirectoryError</code> and <code>NotADirectoryError</code> are all subclasses of it, so catching <code>OSError</code> catches any of them at once when a more specific handler is not needed.

Part 19 covers file handling exceptions in extensive practical depth, including realistic examples combining several of these at once.

3.8 The `RuntimeError` Family

3.8.1 `RecursionError`

```
def countdown(n):
    return countdown(n - 1)

countdown(5)
```

Code Line	Explanation
Definition	A subclass of <code>RuntimeError</code> raised when the maximum recursion depth (Python's built-in limit on how deeply functions can call themselves, by default 1000) is exceeded, almost always because a

Code Line	Explanation
	recursive function is missing a correct base case.
Error output	RecursionError: maximum recursion depth exceeded
Common causes	A recursive function with no base case at all (as above); a base case that is never actually reached due to a logic error in how the recursive argument changes each call.
Prevention	Always verify a recursive function has a correct, reachable base case; for problems that might legitimately require deep recursion, consider rewriting the logic iteratively instead.

3.8.2 NotImplementedError

```
class BaseModel:
    def predict(self, data):
        raise NotImplementedError('Subclasses must implement predict()')

model = BaseModel()
model.predict([1, 2, 3])
```

Unlike the other exceptions in this part, `NotImplementedError` is typically raised deliberately by the programmer, not triggered by an unexpected runtime condition. It signals that a method is intentionally left unimplemented in a base class and must be overridden by a subclass, a common and idiomatic pattern in object-oriented Python design, explored further in Part 24.

3.9 MemoryError

Raised when an operation runs out of available memory. This is comparatively rare in typical application code but can occur when working with very large datasets (loading an enormous file entirely into memory at once) in data science and machine learning contexts. Prevention typically involves processing data in smaller chunks or streams rather than loading everything at once, a strategy discussed further in Part 20.

3.10 AssertionError

```
assert 2 + 2 == 5, 'Math is broken'
```

Raised by the `assert` statement when its condition evaluates to `False`. `AssertionError` has its own complete, dedicated treatment in Part 15, including when assertions are and are not the appropriate tool compared to raising an ordinary exception.

3.11 The Iteration Family: StopIteration and StopAsyncIteration

```
numbers = iter([1, 2])
print(next(numbers))
print(next(numbers))
print(next(numbers))
```

Code Line	Explanation
StopIteration definition	Raised by an iterator's <code>__next__</code> method to signal that there are no more items to produce; calling <code>next()</code> on an exhausted iterator (as in the third call above) raises it.
Why it usually goes unnoticed	A <code>for</code> loop catches <code>StopIteration</code> internally and automatically, on your

Code Line	Explanation
	behalf, which is precisely how for loops know when to stop; it is almost never something application code needs to catch directly, except when manually driving an iterator with <code>next()</code> .
<code>StopAsyncIteration</code>	The asynchronous counterpart, raised by an async iterator's <code>__anext__</code> method and caught automatically by an async for loop, following an identical pattern in async code.

3.12 EOFError, KeyboardInterrupt, and SystemExit

Code Line	Explanation
<code>EOFError</code>	Raised by <code>input()</code> when it hits an end-of-file condition without reading any data, typically when input has been redirected from an already-exhausted file or pipe rather than an interactive terminal.
<code>KeyboardInterrupt</code>	Raised when the user manually interrupts execution, most commonly by pressing Ctrl+C in a terminal. It deliberately does NOT inherit from <code>Exception</code> (see Part 11), specifically so that a broad <code>except Exception: block</code> will not accidentally swallow a user's explicit request to stop the program.
<code>SystemExit</code>	Raised by a call to <code>sys.exit()</code> , used to terminate a program with a specific exit status code. Like <code>KeyboardInterrupt</code> , it deliberately does not inherit from <code>Exception</code> , so that ordinary exception handlers do not accidentally intercept and prevent an intentional program exit.

4. The try Statement

4.1 What try Does

The try statement marks a block of code that Python should monitor for exceptions. If every line inside the try block executes without raising an exception, Python simply continues past the whole try structure normally. If an exception is raised at any point inside the block, Python immediately stops executing the rest of that block and looks for a matching except clause to handle it, rather than letting the program crash outright.

4.2 Basic Syntax

```
try:
    risky_operation()
except SomeException:
    handle_it()
```

Code Line	Explanation
try:	Begins the protected block; every statement indented underneath it is monitored for exceptions.
risky_operation()	The code that might fail; execution proceeds through this normally unless and until something raises an exception.
except SomeException:	Declares a handler for a specific exception type; only runs if that exact type (or a subclass of it) is raised somewhere inside the try block above.

4.3 What Belongs Inside try

- Only the specific operation(s) that can realistically raise the exception(s) you intend to handle, for example the single line that opens a file, or the single line that parses user input.
- Code where you have a genuine, specific plan for what to do if it fails.

4.4 What Should Not Unnecessarily Be Placed Inside try

- Large stretches of unrelated code that happen to follow the risky operation, especially code that cannot itself raise the exception(s) being handled.
- Code you have no real recovery plan for; wrapping something in try/except only to silently pass is usually worse than letting the exception propagate (see Part 26).
- Entire function bodies or entire scripts wrapped in one enormous try block 'just in case', a pattern examined critically in section 4.5 below.

4.5 Why Overly Broad try Blocks Make Debugging Difficult

```
try:
    data = fetch_data(url)
    cleaned = clean_data(data)
    model = train_model(cleaned)
    save_model(model, 'model.pkl')
```

```
except Exception as e:  
    print(f'Something went wrong: {e}')
```

This pattern is tempting because it 'just works' in the sense that the program never crashes, but it creates several real problems. First, when an exception does occur, there is no way to tell from the except block alone which of the four very different operations actually failed; a network failure in `fetch_data()`, a data quality issue in `clean_data()`, a training failure in `train_model()`, and a disk permission issue in `save_model()` would all produce the exact same generic message. Second, it makes it impossible to respond differently to different kinds of failures, since every exception, no matter its type or cause, funnels into the same single handler. Third, it can accidentally suppress exceptions that had nothing to do with the intended risky operation at all, including genuine programming bugs elsewhere in that stretch of code, hiding them behind a generic error message instead of surfacing them clearly.

```
try:  
    data = fetch_data(url)  
except requests.RequestException as e:  
    logger.error(f'Failed to fetch data from {url}: {e}')  
    raise  
  
try:  
    cleaned = clean_data(data)  
except DataValidationError as e:  
    logger.error(f'Data cleaning failed: {e}')  
    raise
```

Splitting the same pipeline into narrower, purpose-specific try blocks, each catching only the exception types genuinely relevant to that specific step, makes the failure point immediately obvious, allows each step to respond appropriately to its own specific failure modes, and keeps unrelated code fully outside the reach of a handler that was never meant to catch its errors.

5. except

5.1 Basic except

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print('Cannot divide by zero')
```

This is the simplest form: catching exactly one specific exception type. Only a `ZeroDivisionError` (or a subclass of it) raised inside the try block will trigger this except clause; any other exception type would not be caught here and would propagate normally.

5.2 Catching Specific Exceptions

Catching the most specific exception type that accurately describes the failure is the generally recommended default, since it ensures the handler only activates for the exact situation it was written to address, and lets every other, unanticipated exception type propagate and surface clearly rather than being silently absorbed.

5.3 Multiple Exception Types in One except

```
try:
    value = int(user_input) / divisor
except (ValueError, ZeroDivisionError) as e:
    print(f'Invalid calculation: {e}')
```

Code Line	Explanation
<code>except (ValueError, ZeroDivisionError) as e:</code>	A single except clause can handle several distinct exception types at once by grouping them in a parenthesized tuple; this except block runs if either type (or a subclass of either) is raised, and in both cases <code>e</code> is bound to whichever specific exception object was actually raised.

5.4 Multiple except Blocks

```
try:
    value = int(user_input)
    result = 100 / value
except ValueError:
    print('Please enter a valid whole number')
except ZeroDivisionError:
    print('Please enter a number other than zero')
```

When different exception types genuinely warrant different responses, as here, use separate except blocks rather than grouping them together; Python checks each except clause in order, from top to bottom, and runs the first one whose exception type matches, then skips every remaining except clause entirely.

5.5 Exception Ordering: Specific Before General

```
try:
```

```

    process_data(data)
except LookupError:
    print('A lookup failed (missing key or index)')
except Exception:
    print('Some other error occurred')

```

Code Line	Explanation
Why order matters	Python checks except clauses top to bottom and uses the first match; since <code>KeyError</code> and <code>IndexError</code> are both subclasses of <code>LookupError</code> , and <code>LookupError</code> is itself a subclass of <code>Exception</code> , a more general clause placed before a more specific one would catch everything first, making the more specific clause below it completely unreachable.
The rule	Always order except clauses from most specific to most general, top to bottom; Python actually raises a <code>SyntaxError</code> in some cases (unreachable except) if this rule is violated too obviously in modern versions, but it is essential to follow deliberately regardless.

5.6 Catching Parent Exceptions

```

try:
    value = my_dict[my_list[0]]
except LookupError as e:
    print(f'Lookup failed: {type(e).__name__}: {e}')

```

Because both `KeyError` and `IndexError` inherit from `LookupError`, a single `except LookupError` clause catches either one, which is useful when the appropriate response genuinely does not depend on which specific kind of lookup failed. `type(e).__name__` retrieves the exact runtime type name of whichever specific exception was actually caught, which is a useful technique when a broader handler still wants to report the precise underlying type.

5.7 except Exception

```

try:
    run_untrusted_plugin(plugin)
except Exception as e:
    logger.error(f'Plugin {plugin.name} failed: {e}')

```

`except Exception`: catches essentially every exception an application should normally ever need to handle, while still deliberately excluding `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit` (see Part 11), since those represent intentional control flow (a user stopping the program, a deliberate exit call) rather than application errors. This is a legitimate, sometimes necessary pattern, most often seen at a genuine top-level boundary, such as a plugin system, a web request handler, or a background worker loop, where an unknown, untrusted, or highly variable piece of code is being executed and the surrounding system must not crash no matter what specific exception that code might raise.

5.8 Why Bare except: Should Generally Be Avoided

```

try:
    run_task()
except:

```

```
print('Something went wrong')
```

Code Line	Explanation
The problem	A bare except: (with no exception type at all) catches literally everything, including KeyboardInterrupt and SystemExit, meaning a user pressing Ctrl+C to stop a runaway program could be silently swallowed and ignored, and a deliberate sys.exit() call elsewhere in the code could be intercepted and prevented from actually exiting.
Additional problems	It also catches genuine programming bugs, like a NameError from a typo, hiding them behind a generic message instead of surfacing them where they can be noticed and fixed.
The fix	Use except Exception: at minimum, which excludes KeyboardInterrupt and SystemExit, or better, catch the specific exception types that are actually anticipated.
Important Distinction: There is a real, narrow difference between except: and except Exception:, and it matters. Bare except: is almost never the right choice in application code; if a truly catch-all handler is needed, except Exception: is the safer minimum.	

6. else

6.1 try/except/else

```
try:
    value = int(user_input)
except ValueError:
    print('Invalid number')
else:
    print(f'You entered {value}, which is valid')
    process(value)
```

The else clause runs only if the try block completed entirely without raising any exception at all. It is optional, and when present it must come after every except clause and before any finally clause.

6.2 When else Executes

Code Line	Explanation
try succeeds, no exception raised	The else block runs, immediately after the try block finishes.
try raises an exception that is caught	The matching except block runs; the else block is skipped entirely.
try raises an exception that is not caught	Neither except nor else runs; the exception propagates upward immediately.

6.3 Why else Improves Clarity

```
# Without else: process(value) is technically still 'inside' the risk zone visually
try:
    value = int(user_input)
    process(value)
except ValueError:
    print('Invalid number')

# With else: intent is explicit; process(value) clearly only runs on success
try:
    value = int(user_input)
except ValueError:
    print('Invalid number')
else:
    process(value)
```

Placing `process(value)` inside the else block, rather than at the end of the try block, makes a precise and important distinction: if `process(value)` itself happens to raise a `ValueError` for some unrelated internal reason, that would be incorrectly caught and misreported as 'Invalid number' in the first version, since it is still physically inside the try block. In the second version, `process(value)` lives in else, which only runs after `int(user_input)` has already succeeded, so if `process()` itself then raises a `ValueError`, it will not be swallowed by the `except ValueError:` clause meant only for the conversion step, and will propagate as a genuinely separate, correctly attributed failure.

6.4 The Difference Between Code in try and Code in else

This is the single most important idea in this part: code inside try is being actively monitored for the specific exception(s) named in the except clauses below it. Code inside else is only reached after the try block has already fully succeeded, and is not monitored by those same except clauses at all. Using else to hold code that depends on the try block's success, but that should not itself be protected by the same exception handling, keeps the scope of each except clause precise and prevents accidentally catching and misattributing an unrelated exception.

7. finally

7.1 What finally Does

The finally clause defines a block of code that always runs, no matter what happens in the try block above it: whether the try block succeeds completely, raises an exception that gets caught, raises an exception that does not get caught, or even executes a return, break, or continue statement partway through. This guarantee makes finally the correct place for cleanup code that absolutely must run regardless of outcome.

7.2 Guaranteed Cleanup Behavior

```
def read_first_line(path):
    f = open(path)
    try:
        return f.readline()
    finally:
        f.close()
        print('File closed')
```

Code Line	Explanation
f.close() inside finally	Guarantees the file handle is always released, whether readline() succeeds, raises an exception, or the function returns early from inside the try block, as it does here.
Why this matters	Leaving files, database connections, or network sockets open unintentionally, known as a resource leak, can exhaust system resources over time in a long-running program; finally is the language's core mechanism for preventing this.

7.3 try/except/finally

```
try:
    process_file(path)
except FileNotFoundError:
    print('File not found')
finally:
    print('Cleanup complete')
```

finally runs after the try block and after whichever except block (if any) handled the exception, regardless of whether an exception occurred at all. If FileNotFoundError is raised, both the except block and then the finally block run, in that order.

7.4 try/else/finally

```
try:
    connection = open_connection()
except ConnectionError:
    print('Could not connect')
else:
    print('Connected successfully')
```

```

    use_connection(connection)
finally:
    print('Cleanup complete')

```

All four keywords can combine: try, a single except, an else, and a finally together. finally still always runs last, regardless of which of the preceding branches (except or else) actually executed.

7.5 try/except/else/finally: The Complete Structure

```

try:
    data = fetch_data()
except APIError as e:
    print(f'API call failed: {e}')
    data = None
else:
    print('Data fetched successfully')
finally:
    print('Request attempt finished')

```

This is the fullest form of the try statement Python supports. The complete execution flow logic for every possible combination of these four blocks is laid out systematically with diagrams in Part 8.

7.6 finally for Files, Database Connections, and Network Resources

```

conn = database.connect()
try:
    conn.execute('UPDATE accounts SET balance = balance - 100 WHERE id = 1')
    conn.commit()
except DatabaseError:
    conn.rollback()
    raise
finally:
    conn.close()

```

This pattern, commit on success, rollback on failure, always close in finally, is the standard, idiomatic structure for any resource that must be released deterministically: database connections, file handles, network sockets, and locks all follow this same shape. Part 23 revisits this exact pattern through context managers (the with statement), which automate it even more cleanly for objects that support it.

7.7 Edge Cases: return, break, and continue Inside try/finally

```

def get_value():
    try:
        return 'from try'
    finally:
        print('finally still runs even though try already returned')

print(get_value())

```

Code Line	Explanation
Behavior	Even though return inside the try block would normally exit the function immediately, Python guarantees the finally block still runs

Code Line	Explanation
	first, before the function actually returns control to the caller; the output prints the finally message, then 'from try'.

```
def confusing_return():
    try:
        return 'from try'
    finally:
        return 'from finally'

print(confusing_return())
```

Code Line	Explanation
A genuine trap	If finally itself also contains a return statement, that return silently overrides the one from try (or even suppresses a pending exception entirely); this function prints 'from finally', not 'from try'. This is considered a serious anti-pattern precisely because it can silently discard an exception that was in the process of propagating, with no visible indication that anything was suppressed.

```
for item in items:
    try:
        process(item)
        if item == 'stop':
            break
    finally:
        print(f'Finished attempt for {item}')
```

The same guarantee applies to break and continue used inside a try block within a loop: finally still runs before the loop actually breaks or continues to the next iteration, ensuring cleanup happens even when a loop is being exited or skipped early from inside a protected block.

Critical Warning: Never put a return, break, or continue statement inside a finally block. Doing so can silently swallow an exception that was actively propagating, without any error or warning, making it exceptionally difficult to debug. Reserve finally strictly for cleanup side effects, not control flow.

8. Complete Exception Handling Flow

This part consolidates exactly which blocks run, and in what order, for every meaningful combination of try, except, else, and finally, so the rules from Parts 4 through 7 can be seen together as one coherent system.

8.1 try -> except

Code Line	Explanation
No exception raised	try runs completely; except is skipped entirely.
Matching exception raised	try runs up to the failing line; except runs; execution continues after the whole try/except structure.
Non-matching exception raised	try runs up to the failing line; except is skipped (it does not match); the exception propagates upward immediately, out of this try structure entirely.

8.2 try -> else

Code Line	Explanation
No exception raised	try runs completely; else runs immediately afterward.
Exception raised (any type, with no except present to catch it)	try runs up to the failing line; else is skipped; the exception propagates upward immediately.

8.3 try -> finally (no except)

Code Line	Explanation
No exception raised	try runs completely; finally always runs afterward; execution continues normally.
Exception raised	try runs up to the failing line; finally still runs; then the exception continues propagating upward, exactly as if finally were not there at all, except that its code did run first.

8.4 try -> except -> finally

Code Line	Explanation
No exception raised	try runs completely; except is skipped; finally runs.
Matching exception raised	try runs up to the failing line; except runs; finally runs; execution continues after the whole structure.
Non-matching exception raised	try runs up to the failing line; except is skipped; finally still runs; then the exception propagates upward.

8.5 try -> except -> else -> finally (the complete structure)

Code Line	Explanation
No exception raised	try runs completely -> except skipped -> else runs -> finally runs.
Matching exception raised	try runs up to the failing line -> matching except runs -> else is skipped (it only runs after a fully successful try) -> finally runs.
Non-matching exception raised	try runs up to the failing line -> no except matches -> else is skipped -> finally still runs -> exception propagates upward.
Exception raised inside the	try runs completely -> except skipped -> else begins running and then

Code Line	Explanation
else block itself	raises -> finally still runs -> that new exception propagates upward (note: the except clauses above do NOT get a second chance to catch an exception raised inside else, since they only monitor the try block).
Unifying Rule: A precise mental model that covers every case above: finally is the only block truly guaranteed to run under all circumstances. else only runs after a completely successful try. except only runs for a matching exception raised specifically inside try (never inside else). Keeping these three boundaries distinct resolves nearly every point of confusion about execution order.	

9. Raising Exceptions with raise

9.1 What raise Does

The raise statement deliberately triggers an exception at a specific point in the code, either a built-in exception or a custom one (Part 10). It is the mechanism by which a program signals 'this specific situation is an error and should not be allowed to continue normally', whether that situation was detected through validation logic, a failed precondition, or an explicit business rule.

9.2 Raising Built-in Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError('Age cannot be negative')
    return age

set_age(-5)
```

Code Line	Explanation
raise ValueError('Age cannot be negative')	Creates a new ValueError instance with the given message and immediately raises it, halting normal execution at that point exactly as if Python itself had detected the problem.
Choosing which built-in to raise	Pick the exception type whose established meaning best matches the situation: ValueError for a value of the right type but wrong content, TypeError for a value of the wrong type entirely, and so on, following the definitions from Part 3.

9.3 Raising Exceptions With Messages

A clear, specific message is one of the most valuable and most frequently neglected parts of raising an exception. raise ValueError('Age cannot be negative') is dramatically more useful to whoever encounters the resulting traceback than a bare raise ValueError(), which provides no context at all about what specifically went wrong.

```
def set_age(age):
    if age < 0:
        raise ValueError(f'Age cannot be negative, got {age}')
    if age > 150:
        raise ValueError(f'Age {age} is not realistic')
    return age
```

Including the actual offending value in the message, as shown here with an f-string, is a small habit that saves significant debugging time later, since the message alone (visible in logs or a traceback) often tells the whole story without needing to reproduce the failure.

9.4 Raising Based on Conditions

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError('Withdrawal amount must be positive')
```

```

if amount > balance:
    raise ValueError('Insufficient funds for this withdrawal')
return balance - amount

```

This is the most common practical pattern for raise: validating a set of preconditions at the top of a function, each with a specific, targeted raise, before proceeding to the function's main logic. This pattern, called fail-fast validation, is examined in depth as a design pattern in Part 29.

9.5 Re-Raising Exceptions (Preview)

```

try:
    risky_operation()
except ValueError:
    print('Logging the problem...')
    raise

```

A bare raise with no arguments, used inside an except block, re-raises the exact exception that is currently being handled, preserving its original type, message, and traceback. This is introduced briefly here because it uses the same raise keyword, but it has its own complete, dedicated treatment in Part 14, including when re-raising is the correct choice versus letting an exception propagate untouched in the first place.

9.6 Practical Validation Example

```

def create_user_account(username, email, age):
    if not username or not username.strip():
        raise ValueError('Username cannot be empty')
    if '@' not in email:
        raise ValueError(f'Invalid email address: {email}')
    if not isinstance(age, int):
        raise TypeError(f'Age must be an integer, got {type(age).__name__}')
    if age < 13:
        raise ValueError('Users must be at least 13 years old')
    return {'username': username, 'email': email, 'age': age}

create_user_account('ada', 'not-an-email', 25)

```

Code Line	Explanation
Overall pattern	Each precondition is checked independently, and each raises the most specific, appropriate built-in exception type with a message describing exactly what was wrong and, where useful, what value caused the failure.
TypeError vs ValueError usage here	The age type check uses TypeError, since receiving a string or float where an integer was required is a fundamentally wrong kind of value; every other check uses ValueError, since those values are the correct type but an unacceptable specific value, following the distinction drawn in Part 3.4.

10. Custom Exceptions

10.1 Why Create Custom Exceptions

Built-in exceptions like `ValueError` and `TypeError` are general-purpose and describe categories of problems, not specific business or domain meaning. A custom exception lets code express a specific, named failure condition that is meaningful within its own domain, such as `InsufficientFundsError` for a banking system or `ModelNotTrainedError` for a machine learning pipeline, which is far more informative and far easier to catch selectively than a generic `ValueError` with a similar message.

10.2 Creating a Custom Exception Class

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(
            f'Cannot withdraw {amount}: balance is only {balance}'
        )
    return balance - amount
```

Code Line	Explanation
<code>class InsufficientFundsError(Exception):</code>	Defines a brand new exception type by inheriting from <code>Exception</code> (never from <code>BaseException</code> directly, as explained in Part 11); this single line is enough to create a fully functional, raisable, catchable exception.
<code>pass</code>	The class needs no additional behavior beyond what <code>Exception</code> already provides (storing a message, supporting <code>str()</code> , and so on), so its body can simply be <code>pass</code> .
<code>raise InsufficientFundsError(f'...')</code>	Used exactly like any built-in exception; the message passed in becomes accessible the same way a built-in exception's message would be.

10.3 Custom Messages and Custom `__init__`

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        self.shortfall = amount - balance
        message = f'Need {self.shortfall} more to withdraw {amount} (balance: {balance})'
        super().__init__(message)

try:
    raise InsufficientFundsError(balance=50, amount=200)
except InsufficientFundsError as e:
    print(e)
```

```
print(e.shortfall)
```

Code Line	Explanation
<code>def __init__(self, balance, amount):</code>	Overrides the default constructor to accept meaningful, structured arguments specific to this exception's domain, rather than only a single generic message string.
<code>self.balance = balance / self.amount = amount / self.shortfall = ...</code>	Stores each piece of information as a custom attribute on the exception instance itself, making it directly accessible to any code that catches this exception later, not just visible as text inside a message.
<code>super().__init__(message)</code>	Calls the parent Exception class's own <code>__init__</code> with a fully composed, human-readable message; this is what makes <code>str(e)</code> and printing the exception produce sensible output, and should almost always be called to preserve normal exception behavior.
<code>e.shortfall</code>	Because <code>shortfall</code> was stored as an attribute, code that catches this exception can access the precise numeric <code>shortfall</code> directly, enabling a much more specific and useful response than parsing it back out of a text message.

10.4 Multiple Custom Exceptions and Exception Hierarchies

```
class BankingError(Exception):
    '''Base class for all banking-related errors.'''
    pass

class InsufficientFundsError(BankingError):
    pass

class InvalidAmountError(BankingError):
    pass

class AccountFrozenError(BankingError):
    pass

def withdraw(account, amount):
    if account.frozen:
        raise AccountFrozenError(f'Account {account.id} is frozen')
    if amount <= 0:
        raise InvalidAmountError('Withdrawal amount must be positive')
    if amount > account.balance:
        raise InsufficientFundsError('Not enough funds')
```

Code Line	Explanation
<code>class BankingError(Exception):</code>	A custom base class specific to this application's domain, itself inheriting directly from <code>Exception</code> .
<code>class InsufficientFundsError(BankingError): / etc.</code>	Each specific error inherits from <code>BankingError</code> rather than directly from <code>Exception</code> , forming a small custom hierarchy that mirrors the built-in exception hierarchy's own design (see Part 11).
Why this matters when catching	Calling code can now choose its level of specificity: except <code>InsufficientFundsError</code> : to handle only that one exact situation, or except <code>BankingError</code> : to handle any banking-related problem

Code Line	Explanation
	uniformly, without needing to list every specific subclass individually.

10.5 A Gallery of Realistic Domain-Specific Custom Exceptions

```
class InvalidAgeError(ValueError):
    '''Raised when an age value fails domain validation.'''
    pass

class InvalidTransactionError(Exception):
    '''Raised when a financial transaction violates business rules.'''
    pass

class DataValidationError(Exception):
    '''Raised when a dataset fails a validation check.'''
    def __init__(self, message, column=None):
        self.column = column
        super().__init__(message)

class ModelNotTrainedError(Exception):
    '''Raised when prediction is attempted before training.'''
    pass

class InvalidConfigurationError(Exception):
    '''Raised when application configuration is invalid or incomplete.'''
    pass
```

Code Line	Explanation
class InvalidAgeError(ValueError):	Demonstrates inheriting from a built-in exception instead of Exception directly; this is a deliberate, useful choice, since code that only knows to catch the generic ValueError will still catch InvalidAgeError automatically, while code that specifically wants to catch InvalidAgeError still can.
DataValidationError with a column attribute	Shows a custom exception carrying structured, domain-specific data (which column failed validation) beyond just a text message, directly usable by whatever code catches it, following the same pattern as section 10.3.
ModelNotTrainedError, InvalidConfigurationError	Both used extensively in the Data Science, Machine Learning, and AI Engineering examples throughout Parts 20 and 21.

10.6 When to Create a Custom Exception

- The failure represents a specific, named concept within your application's own domain, not a generic programming mistake.
- Calling code would benefit from catching this specific failure separately from other, unrelated errors.
- The failure needs to carry structured data beyond a plain text message (like the shortfall or column examples above).

- You are building a library or module that others will use, where a domain-specific exception type provides a clearer, more stable public API than raising generic built-in exceptions for every failure.

11. Exception Hierarchy

11.1 Python's Exception Hierarchy Visualized

```

BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
    +-- ArithmeticError
    |   +-- ZeroDivisionError
    |   +-- OverflowError
    |   +-- FloatingPointError
+-- LookupError
    |   +-- IndexError
    |   +-- KeyError
+-- TypeError
+-- ValueError
+-- AttributeError
+-- NameError
    |   +-- UnboundLocalError
+-- ImportError
    |   +-- ModuleNotFoundError
+-- OSError
    |   +-- FileNotFoundError
    |   +-- PermissionError
    |   +-- IsADirectoryError
    |   +-- NotADirectoryError
+-- RuntimeError
    |   +-- RecursionError
    |   +-- NotImplementedError
+-- MemoryError
+-- AssertionError
+-- StopIteration
+-- StopAsyncIteration
+-- EOFError

```

This diagram, while not exhaustive of every single built-in exception, shows the essential shape of the hierarchy and every exception introduced in Part 3, organized by actual inheritance relationship rather than alphabetically.

11.2 BaseException vs Exception: Why the Split Exists

Code Line	Explanation
BaseException	The true root of everything. Deliberately includes not just application errors but also SystemExit, KeyboardInterrupt, and GeneratorExit, three events that represent intentional control flow or environment-level signals rather than application-level failures.
Exception	A direct child of BaseException that serves as the practical root for essentially all application-level error handling. The overwhelming majority of code should catch Exception or one of its subclasses, never BaseException directly.

Code Line	Explanation
The design reasoning	If except Exception: (or a bare except:) accidentally also caught SystemExit or KeyboardInterrupt, a program would become unable to be cleanly stopped by sys.exit() or Ctrl+C from inside any code with a broad handler; by placing those three specifically outside the Exception branch, Python guarantees that even a reasonably broad except Exception: handler cannot accidentally interfere with intentional program termination.

11.3 Why KeyboardInterrupt, SystemExit, and GeneratorExit Are Special

Code Line	Explanation
KeyboardInterrupt	Raised by the interpreter when the user sends an interrupt signal (typically Ctrl+C). It represents an external, human decision to stop the program, not an internal application error, so it should almost always be allowed to propagate and actually stop execution.
SystemExit	Raised by sys.exit(); represents a deliberate, intentional request from the code itself to terminate the whole program with a specific exit status. Catching it broadly would defeat the very purpose of calling sys.exit() in the first place.
GeneratorExit	Raised inside a generator function when its close() method is called or it is being garbage collected while still suspended; it signals the generator to clean up and stop, not that something has gone wrong within the generator's own logic.

11.4 Why Inheritance Matters When Catching Exceptions

```
try:
    value = my_dict['missing_key']
except LookupError:
    print('Caught it via the parent class LookupError')
```

This works because KeyError is a subclass of LookupError; except LookupError: catches KeyError, IndexError, or any other LookupError subclass automatically, without needing to name each one individually. This is the entire practical reason the hierarchy exists in the shape it does: it lets calling code choose exactly how broad or narrow its handling should be, from one extremely specific exception type up to a wide family, simply by choosing which level of the hierarchy to catch.

11.5 Checking Hierarchy Relationships Directly

```
print(issubclass(KeyError, LookupError))
print(issubclass(KeyError, Exception))
print(issubclass(KeyboardInterrupt, Exception))
print(KeyError.__mro__)
```

Code Line	Explanation
issubclass(KeyError, LookupError)	Produces True, confirming the relationship shown in the hierarchy diagram directly, in code.
issubclass(KeyboardInterrupt, Exception)	Produces False, directly confirming that KeyboardInterrupt sits outside the Exception branch, exactly as explained in section 11.3.

Code Line	Explanation
<code>KeyError.__mro__</code>	MRO stands for Method Resolution Order; this attribute lists every class in <code>KeyError</code> 's full inheritance chain, from <code>KeyError</code> itself up through <code>LookupError</code> , <code>Exception</code> , <code>BaseException</code> , and finally the universal base object, showing the complete hierarchy path for this one specific exception type.

12. Exception Objects

12.1 Exceptions Are Objects

Every exception in Python, whether built-in or custom, is an ordinary object: a specific instance of a specific class. `raise ValueError('bad input')` does exactly two things: it constructs a new `ValueError` instance (calling `ValueError.__init__` with the given message, precisely like calling any other class), and then it raises that instance. This is why exceptions can carry data, support methods, and participate fully in Python's normal object-oriented features, including custom `__init__` methods and custom attributes, as already demonstrated in Part 10.

12.2 `except ... as e`: What `e` Represents

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(type(e))
    print(e)
    print(e.args)
```

Code Line	Explanation
<code>except ZeroDivisionError as e:</code>	Binds the name <code>e</code> to the actual exception object that was raised and caught, for the remainder of the <code>except</code> block; <code>e</code> is not the exception class itself, it is one specific instance of it, carrying whatever data was attached when it was raised.
<code>type(e)</code>	Confirms the exact runtime type of the caught exception, here <code><class 'ZeroDivisionError'></code> ; useful when a broader <code>except</code> clause (like <code>except Exception</code>) needs to know precisely which specific type was actually caught.
<code>print(e)</code>	Implicitly calls <code>str(e)</code> (see 12.4), producing the human-readable message: <code>'division by zero'</code> .
<code>e.args</code>	Every exception stores the arguments it was constructed with as a tuple in <code>.args</code> ; here, <code>('division by zero',)</code> , a single-element tuple containing the message.

12.3 Exception Messages and `.args`

```
class InvalidTransactionError(Exception):
    pass

try:
    raise InvalidTransactionError('Amount', -50, 'is invalid')
except InvalidTransactionError as e:
    print(e.args)
    print(str(e))
```

Code Line	Explanation
<code>raise InvalidTransactionError('Amount', -50, 'is invalid')</code>	An exception's constructor can accept any number of positional arguments, not only a single message string; all of them are stored

Code Line	Explanation
	together in .args.
e.args	Prints ('Amount', -50, 'is invalid'), the exact tuple of arguments passed to the constructor, preserved and accessible after the exception is caught.
str(e)	When more than one argument is passed, the default str() representation prints the whole args tuple rather than a single clean message, which is why exceptions expecting multiple pieces of information typically compose them into one formatted message string instead, as shown in Part 10.3.

12.4 str(exception) vs repr(exception)

```
try:
    raise ValueError('invalid input')
except ValueError as e:
    print(str(e))
    print(repr(e))
```

Code Line	Explanation
str(e)	Produces the plain, human-readable message: invalid input. This is what print(e) and an f-string like f'{e}' both use automatically.
repr(e)	Produces a more complete, unambiguous, developer-facing representation that also names the exception type: ValueError('invalid input'). This is more useful in logs or debugging output where knowing the exact type matters, not just the message text.

12.5 Accessing Exception Information Broadly

```
import sys

try:
    risky_operation()
except Exception as e:
    exc_type, exc_value, exc_tb = sys.exc_info()
    print(exc_type)
    print(exc_value)
    print(exc_tb.tb_lineno)
```

Code Line	Explanation
sys.exc_info()	Returns a tuple of the exception type, the exception instance, and the traceback object for the exception currently being handled; useful in generic logging or error-reporting utilities that need traceback details without the exception itself being explicitly bound with as e.
exc_tb.tb_lineno	The traceback object exposes the exact line number where the exception occurred, among other structured details, which is the same underlying information Python uses to construct the printed traceback text described in Part 2.

Practical Guidance: In modern, everyday code, except Exception as e: combined with e.args, str(e), and

Code Line	Explanation
	<code>type(e).__name__</code> covers the vast majority of practical needs; <code>sys.exc_info()</code> is mainly reserved for lower-level tooling like custom logging handlers or frameworks.

13. Exception Chaining

13.1 Why Exception Chaining Exists

When one exception is raised while another exception is already being handled, or deliberately raised as a translated version of an underlying failure, Python needs a way to preserve the connection between the two so nothing is lost. Exception chaining is exactly this mechanism: it keeps a link from the new exception back to the original one, so a traceback shows the full causal story, not just the final, possibly less informative symptom.

13.2 Implicit Exception Chaining

```
try:
    result = 10 / 0
except ZeroDivisionError:
    raise ValueError('Calculation failed')
```

```
Traceback (most recent call last):
  File "app.py", line 2, in <module>
    result = 10 / 0
    ~~~^~~
ZeroDivisionError: division by zero
```

ZeroDivisionError: division by zero

During handling of the above exception, another exception occurred:

```
Traceback (most recent call last):
  File "app.py", line 4, in <module>
    raise ValueError('Calculation failed')
ValueError: Calculation failed
```

Code Line	Explanation
What happened	Because the ValueError was raised from inside the except block that was actively handling the ZeroDivisionError, Python automatically links the two, without any special syntax being written at all; this is implicit chaining.
'During handling of the above exception, another exception occurred:'	This exact phrase is Python's signal that implicit chaining occurred; both tracebacks are shown, in order, so the full picture, the original division failure and the resulting ValueError, is visible together.
__context__	Internally, Python stores this automatic link in the new exception's __context__ attribute, pointing back to the exception that was being handled at the time.

13.3 Explicit Exception Chaining With raise ... from ...

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    raise ValueError('Calculation failed') from e
```

```
Traceback (most recent call last):
  File "app.py", line 2, in <module>
    result = 10 / 0
    ~~~~^~~~
```

ZeroDivisionError: division by zero

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "app.py", line 4, in <module>
    raise ValueError('Calculation failed') from e
ValueError: Calculation failed
```

Code Line	Explanation
raise ValueError(...) from e	The from e explicitly declares e as the direct cause of the new exception, rather than relying on Python's automatic implicit link; this is a deliberate statement of intent by the programmer, not merely an accidental byproduct of where the raise happens to be located.
'The above exception was the direct cause of the following exception:'	This distinct phrase (different from the implicit chaining phrase in 13.2) confirms explicit chaining was used; Python stores this link in the new exception's <code>__cause__</code> attribute.
Why prefer explicit chaining	It communicates clear intent to future readers of the code, and it takes priority for display purposes over any implicit context that might also exist, giving the exact same resulting traceback regardless of exactly where in a broader try block the raise happened to occur.

13.4 `__cause__`, `__context__`, and `__suppress_context__`

Code Line	Explanation
<code>__cause__</code>	Set only by explicit raise ... from ... chaining; represents a deliberate, programmer-declared causal relationship between two exceptions.
<code>__context__</code>	Set automatically by Python whenever a new exception is raised while another is already being handled, regardless of whether from was used; represents what was simply happening at the time, not necessarily a deliberate causal claim.
<code>__suppress_context__</code>	Automatically set to True whenever raise ... from ... is used; this tells Python's default traceback printer to display only the explicit <code>__cause__</code> chain and hide the (potentially less relevant) automatic <code>__context__</code> , keeping the printed traceback focused and uncluttered.

13.5 Suppressing Chaining Entirely With raise ... from None

```
try:
    value = int(user_input)
except ValueError:
    raise ValueError('Please enter a valid whole number') from None
```

Using from None explicitly discards any chaining link entirely, producing a clean traceback that shows only the new exception, with no 'During handling of...' or 'The above exception was the direct cause...' preamble at all. This is useful in situations, like validating user-facing input, where the low-level internal ValueError (the exact

parsing failure) adds no real value to the person reading the final error message, and showing it would only be noise.

13.6 Why Exception Chaining Matters in Professional Applications

```
class DataProcessingError(Exception):
    pass

def process_record(record):
    try:
        return record['value'] / record['divisor']
    except (KeyError, ZeroDivisionError) as e:
        raise DataProcessingError(f'Failed to process record {record}') from e
```

This is exception translation, a design pattern covered fully in Part 29: converting a low-level, implementation-specific exception (`KeyError`, `ZeroDivisionError`) into a higher-level, domain-meaningful one (`DataProcessingError`) that is more useful to code calling `process_record()`. Using `from e` ensures that, even though the caller now only needs to catch `DataProcessingError`, the original low-level cause is never lost; it remains fully visible in the traceback and accessible programmatically through the exception's `__cause__` attribute, for example during debugging or detailed logging.

14. Re-Raising Exceptions

14.1 Bare raise

```
try:
    connect_to_database()
except ConnectionError:
    print('Connection failed, alerting monitoring system')
    alert_monitoring()
    raise
```

Code Line	Explanation
raise (with no arguments)	When used inside an except block, re-raises the exact exception object currently being handled, with its original type, message, and traceback fully intact, exactly as if this except block had never caught it at all.
Why use it here	The except block performs a genuinely useful side effect (alerting a monitoring system) but does not actually resolve the underlying connection problem; re-raising ensures the caller of this code still sees the failure and can respond to it, rather than the problem being silently absorbed.

14.2 Re-Raising vs Raising a New Exception

```
# Re-raising: preserves the exact original exception
try:
    risky()
except ValueError:
    log_error()
    raise

# Raising a new exception: replaces it entirely (loses original type unless chained)
try:
    risky()
except ValueError as e:
    log_error()
    raise RuntimeError('risky() failed') from e
```

Bare raise is the correct choice when the original exception type and message are still exactly right for the caller to see, and the except block is only adding a side effect (logging, cleanup, an alert) along the way. Raising a genuinely new exception (ideally chained with `from e`, per Part 13) is the correct choice when the original exception's type is too low-level or implementation-specific for the caller, and a translated, higher-level exception communicates the failure more usefully.

14.3 Modifying vs Preserving Exceptions

```
try:
    process_batch(records)
except ValueError as e:
```

```
raise ValueError(f'Batch processing failed: {e}') from e
```

This pattern re-raises the same exception type but with an enriched, more contextual message, chaining back to the original via `from e` so no information is lost. This is a middle ground between a bare `raise` (identical exception) and raising a completely different exception type: it adds valuable context about where and why the failure occurred at a higher level, while keeping the original exception fully traceable underneath it.

14.4 Logging and Re-Raising

```
import logging
logger = logging.getLogger(__name__)

def transfer_funds(from_acct, to_acct, amount):
    try:
        _execute_transfer(from_acct, to_acct, amount)
    except InsufficientFundsError:
        logger.warning(f'Transfer failed: insufficient funds in {from_acct}')
        raise
    except DatabaseError:
        logger.exception('Unexpected database error during transfer')
        raise
```

A very common and professionally important pattern: log the failure with appropriate context and severity right where it is caught (since that is where the most local, relevant information is available), then re-raise so the failure still propagates to wherever the application's higher-level error handling ultimately needs to respond to it, for example returning an HTTP error response or rolling back a larger transaction. Part 25 covers `logger.exception()` and the full logging module in complete depth.

14.5 When Re-Raising Is Appropriate

- The current function or block cannot actually resolve the problem, only observe or react to it (logging, alerting, cleanup).
- The caller genuinely needs to know the operation failed in order to make its own correct decision (retry, abort, report to a user).
- You are deliberately translating a low-level exception into a more meaningful one for your callers (use `raise NewException(...) from e`, not a bare `raise`, in this case).

Conversely, catching an exception and not re-raising it (nor otherwise properly handling the underlying problem) should be a deliberate, justified choice, not a default; Part 26 examines the failure mode of silently swallowing exceptions in detail.

15. Assertions

15.1 The assert Statement

`assert` is a debugging aid built into the language: it checks that a condition is true, and if it is not, immediately raises `AssertionError`. It exists primarily to state and verify internal assumptions during development, not to validate external input or handle expected runtime failure conditions, a distinction explored fully in section 15.6.

15.2 Assertion Syntax

```
assert condition
assert condition, 'Optional message shown if the assertion fails'
```

Code Line	Explanation
<code>assert condition</code>	Evaluates condition; if it is truthy, execution simply continues as if the line were not there at all. If it is falsy, Python raises <code>AssertionError</code> with no message.
<code>assert condition, message</code>	The optional second part, separated by a comma, becomes the <code>AssertionError</code> 's message if the assertion fails, making the failure far more informative than a bare <code>AssertionError</code> with no explanation.

15.3 Assertion Conditions and Messages in Practice

```
def calculate_discount(price, percent):
    assert 0 <= percent <= 100, f'Discount percent must be 0-100, got {percent}'
    return price * (1 - percent / 100)

calculate_discount(100, 150)
```

```
Traceback (most recent call last):
```

```
...
```

```
AssertionError: Discount percent must be 0-100, got 150
```

This is a typical, appropriate use of `assert`: verifying an internal assumption (that a percentage argument is genuinely within a sane range) that should only ever be violated by a programming mistake elsewhere in the same codebase, not by ordinary external circumstances.

15.4 Assertions vs Exceptions

Code Line	Explanation
<code>assert</code>	Intended to catch programming mistakes and verify internal invariants during development and testing; conceptually says 'this should never happen if my code is correct'. Always raises exactly <code>AssertionError</code> .
<code>raise (an ordinary exception)</code>	Intended to signal genuinely expected runtime failure conditions that the calling code should anticipate and potentially handle: invalid user input, a missing file, a failed network request. Uses whichever exception type accurately describes the situation.

15.5 Assertions vs Input Validation

```
# WRONG: using assert to validate untrusted external input
def process_signup(age):
    assert age >= 13, 'Must be at least 13'
    create_account(age)

# RIGHT: using an ordinary exception for external, user-facing validation
def process_signup(age):
    if age < 13:
        raise ValueError('Must be at least 13')
    create_account(age)
```

age coming from user signup input is external, untrusted data, not an internal programming assumption; it can absolutely be invalid through no bug in the code at all (a user simply enters the wrong number), so it must be validated with an ordinary exception. The critical reason this distinction matters practically is explained next in section 15.6: assertions can be silently disabled entirely, which would make this validation vanish completely if written the wrong way.

15.6 When Assertions Should and Should Not Be Used

- Use assert for internal sanity checks and invariants during development: verifying a function's own internal logic produced a value in the expected range, checking that a precondition your own code should have already guaranteed truly holds.
- Use assert liberally in test code (though most testing frameworks provide their own richer assert-style functions, covered outside this handbook's scope).
- Never use assert to validate external input: user input, file contents, API responses, configuration values, or any data originating outside your own program's direct control.
- Never rely on an assertion's side effects for correctness, since (per section 15.7) the entire statement can be stripped out entirely under certain conditions.

15.7 Python Optimization and Disabled Assertions

```
python3 -O myscript.py
```

When Python is run with the `-O` (optimize) command-line flag, every assert statement in the entire program is completely skipped, as if it were never written at all, no matter its condition. This is precisely why assert must never be used for anything the program's correctness genuinely depends on, such as input validation or security checks: code that works correctly in normal testing could silently stop performing those checks entirely in an optimized deployment, with no warning or error of any kind. This single fact is the strongest, most concrete justification for the assert-vs-exception distinction drawn throughout this part.

Critical Warning: A validation check written with assert can vanish entirely in production if the `-O` flag is ever used, without any visible sign that it happened. Any check whose absence would be a security or correctness problem must use an ordinary raise, never assert.

16. Exception Handling with Functions

16.1 Exceptions Raised Inside Functions

```
def divide(a, b):  
    return a / b  
  
result = divide(10, 0)
```

When an exception is raised inside a function and nothing inside that function catches it, the function's execution stops immediately at that point; none of its remaining code runs, including any return statement that might have followed. The exception then propagates out to wherever the function was called from, as detailed fully in Part 17.

16.2 Catching Exceptions Inside a Function

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return None  
  
print(safe_divide(10, 0))  
print(safe_divide(10, 2))
```

A function can handle its own exceptions internally, fully containing the failure so it never escapes to the caller at all. This is appropriate when the function itself has a genuine, sensible way to recover or provide a fallback result, as shown here.

16.3 Letting Exceptions Propagate

```
def load_user_settings(path):  
    with open(path) as f:  
        return json.load(f)
```

This function deliberately does not catch `FileNotFoundError`, `PermissionError`, or `json.JSONDecodeError` at all; it simply lets them propagate to its caller. This is a legitimate and often preferable design choice:

`load_user_settings()` itself has no meaningful way to decide what should happen if the file is missing (should a default be used? should the whole program abort? should the user be prompted?), and that decision genuinely belongs to whichever code is calling it, which has more context about the overall situation.

16.4 Returning Errors vs Raising Exceptions

```
# Style A: returning a sentinel/error value  
def parse_age(text):  
    if not text.isdigit():  
        return None  
    return int(text)  
  
# Style B: raising an exception  
def parse_age(text):
```

```

if not text.isdigit():
    raise ValueError(f'Invalid age: {text}')
return int(text)

```

Code Line	Explanation
Returning a sentinel value (Style A)	Forces every single caller to remember to explicitly check the return value before using it (if result is None: ...); if a caller forgets this check, the None silently flows further into the program and can cause a confusing failure much later, far from its true origin.
Raising an exception (Style B)	Makes it impossible to accidentally ignore the failure; if a caller does not explicitly handle it, the exception propagates loudly and visibly, with a full traceback pointing directly back to the actual problem, which is generally far easier to debug than a silently wrong None flowing downstream.
General guidance	For a genuine error condition (invalid input, a failed operation), raising an exception is usually the more robust and Pythonic choice. Returning None or a similar sentinel is more appropriate for representing a normal, expected 'no result' outcome that is not really an error at all, such as a search function that legitimately found nothing.

16.5 Exceptions Across Function Calls

```

def get_config_value(config, key):
    return config[key]

def get_database_url(config):
    return get_config_value(config, 'database_url')

def connect(config):
    url = get_database_url(config)
    return create_connection(url)

connect({})

```

Here, a missing 'database_url' key raises a `KeyError` inside `get_config_value()`, which is called by `get_database_url()`, which is itself called by `connect()`. None of these three functions catches the exception, so it travels all the way back through every one of these calls to wherever `connect()` was originally invoked from. This chain of calls, and precisely how an exception travels back through it, is the exact subject of Part 17.

17. Exception Propagation

17.1 How an Exception Moves Through Nested Calls

When a function raises an exception that it does not itself catch, Python does not simply discard the function call and move on; it unwinds the call stack one frame at a time. The current function's execution stops completely, control returns to whichever line called it, and Python checks whether that calling code catches the exception. If not, the same thing happens again one level up, and so on, until either a matching except block is found somewhere in the chain, or the exception reaches the very top of the program with nothing left to catch it, at which point the program terminates and the traceback (Part 2) is printed.

17.2 A Complete Worked Example

```
def operation(value):
    return 100 / value

def validate_data(data):
    for item in data:
        operation(item)

def process_data(data):
    validate_data(data)
    print('Processing complete')

def main():
    dataset = [10, 5, 0, 2]
    process_data(dataset)

main()
```

Code Line	Explanation
Call chain leading to the failure	main() calls process_data(dataset), which calls validate_data(data), which loops through the data and calls operation(item) for each one.
Where the actual failure occurs	When item is 0 (the third element), operation(0) attempts 100 / 0 and raises ZeroDivisionError; this is the deepest point in the call chain, and none of the four functions (operation, validate_data, process_data, main) contains any try/except at all.
What happens next	The exception propagates out of operation(), then out of validate_data() (skipping the remaining loop iterations entirely, so item value 2 is never processed), then out of process_data() (so 'Processing complete' never prints), then out of main(), and finally out of the program itself, since nothing anywhere caught it.

17.3 The Resulting Traceback, Explained

```
Traceback (most recent call last):
  File "pipeline.py", line 15, in <module>
    main()
  File "pipeline.py", line 13, in main
```

```

    process_data(dataset)
File "pipeline.py", line 9, in process_data
    validate_data(data)
File "pipeline.py", line 6, in validate_data
    operation(item)
File "pipeline.py", line 2, in operation
    return 100 / value
        ~~~~^~~~~~

```

ZeroDivisionError: division by zero

This traceback is a direct, literal printout of the exact call chain described in section 17.2: main -> process_data -> validate_data -> operation, read from outermost (top) to innermost (bottom), confirming precisely how the exception traveled backward through four full function calls before finally reaching the top level with nothing to catch it.

17.4 Adding a Handler at the Right Level

```

def process_data(data):
    try:
        validate_data(data)
    except ZeroDivisionError as e:
        print(f'Skipping invalid data: {e}')
    print('Processing complete')

```

Placing the try/except at the process_data level, rather than deep inside operation() itself, is often a deliberate design decision: operation() and validate_data() stay simple and focused purely on their core logic, while the decision about how to actually respond to a failure (here, logging a message and continuing) is made at a level with enough context to decide what 'handling' the error should actually mean for the wider pipeline. Choosing the right level in the call chain to place a handler is a central theme of Part 18 and Part 29.

18. Nested Exception Handling

18.1 Nested try Blocks

```
try:
    print('Outer try starts')
    try:
        result = 10 / 0
    except ValueError:
        print('Inner handler: this will NOT run (wrong exception type)')
        print('This line is skipped, since the inner exception was not caught')
except ZeroDivisionError:
    print('Outer handler: caught it here instead')
```

A try block can be nested entirely inside another try block. If the inner try raises an exception that its own except clauses do not match, the exception propagates out of the inner try structure entirely (exactly as it would out of a function, per Part 17) and is then checked against the outer try's except clauses in turn.

18.2 Inner vs Outer Exception Handling

```
try:
    config = load_config()
    try:
        connection = connect_to_database(config)
    except ConnectionError:
        print('Database unavailable, using cached data')
        connection = None
    run_application(connection)
except FileNotFoundError:
    print('Configuration file missing, cannot start application')
```

Code Line	Explanation
Inner except ConnectionError	Handles a specific, localized, recoverable failure (the database being unreachable) right where it occurs, allowing the rest of the outer block to continue running with a fallback.
Outer except FileNotFoundError	Handles a completely different, more fundamental failure (the configuration file itself being missing) at a broader level, since there is no sensible way to continue at all without configuration.
Why nesting helps here	Each exception is handled at the level of the code that has enough context to respond appropriately; the inner failure has a graceful fallback, while the outer failure does not and should stop the whole startup sequence.

18.3 Propagation From Inner to Outer Blocks

Exactly as with nested function calls (Part 17), an exception raised inside a nested try block that is not caught by that inner try's own except clauses continues propagating outward, checked next against the enclosing try block's except clauses, and so on outward through as many levels of nesting as exist, until it is either caught somewhere or reaches the top of the program entirely uncaught.

18.4 When Nested try Is Genuinely Useful

- Different operations within the same overall block have genuinely different, independent failure modes that call for different specific responses, as in the database/config example in section 18.2.
- An inner operation has a sensible, self-contained fallback or retry that should not abort the entire surrounding operation.
- Resource cleanup for one specific inner resource needs to happen independently of a broader outer operation's own error handling (often better expressed with finally or a context manager, covered in Part 23).

18.5 When Nesting Creates Unnecessary Complexity

```
try:
    try:
        try:
            value = int(data['field'])
        except ValueError:
            raise
        except KeyError:
            raise
    except Exception:
        print('Something failed')
```

Deep, multi-level nesting where each level does nothing more than immediately re-raise, as shown here, adds no real value and only makes the code harder to read and reason about. In nearly every such case, a single flat try block with multiple except clauses (Part 5) expresses exactly the same handling logic far more clearly.

```
try:
    value = int(data['field'])
except (ValueError, KeyError):
    print('Something failed')
```

Design Guidance: A useful rule of thumb: nest try blocks only when the inner and outer blocks genuinely need different, independent handling logic. If every level of nesting ultimately funnels into the same outer response, flatten the structure into a single try block with the appropriate except clauses instead.

19. File Handling Exceptions

19.1 Missing Files

```
def read_report(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        print(f'Report not found at {path}; using empty report')
        return ''
```

`FileNotFoundError` is the single most common file-related exception encountered in practice. Handling it with a clear fallback or a clear user-facing message, rather than letting the program crash with a raw traceback, is standard practice for any file path that originates from configuration, user input, or an external system rather than a path the program controls entirely itself.

19.2 Permission Errors

```
def save_report(path, content):
    try:
        with open(path, 'w') as f:
            f.write(content)
    except PermissionError:
        print(f'Permission denied writing to {path}; check file permissions')
        raise
```

`PermissionError` typically indicates a genuine environment or deployment problem (incorrect file ownership, a read-only filesystem, insufficient user privileges) rather than something the program itself can meaningfully work around; logging a clear diagnostic message and then re-raising (Part 14) so the failure is visible and addressed at the infrastructure level is usually the appropriate response.

19.3 Invalid File Paths

```
def read_data_file(path):
    try:
        with open(path) as f:
            return f.read()
    except IsADirectoryError:
        raise ValueError(f'{path} is a directory, not a file') from None
    except NotADirectoryError:
        raise ValueError(f'A component of {path} is not a directory') from None
```

`IsADirectoryError` and `NotADirectoryError` (introduced in Part 3.7) both indicate a fundamental mismatch between what the code expected at a given path and what actually exists there; translating them into a single, clearer `ValueError` (via exception translation, Part 13.6 and Part 29) is often more useful to callers than exposing these fairly low-level OS-specific distinctions directly.

19.4 Encoding Errors

```
def read_text_file(path):
    try:
        with open(path, encoding='utf-8') as f:
            return f.read()
    except UnicodeDecodeError as e:
        print(f'File {path} is not valid UTF-8 text: {e}')
        with open(path, encoding='latin-1') as f:
            return f.read()
```

Code Line	Explanation
UnicodeDecodeError	Raised when the bytes in a file cannot be interpreted using the specified text encoding; extremely common when working with files of unknown or mixed origin, since not every text file is UTF-8 encoded.
The fallback pattern shown	Attempting a second, more permissive encoding (latin-1 can decode any byte sequence without error, though not always correctly) is one practical strategy; another is to open the file in binary mode and handle decoding manually with explicit error handling (errors='replace' or errors='ignore' passed to open()).

19.5 Invalid or Corrupted Data

```
import csv

def load_csv_safely(path):
    rows = []
    try:
        with open(path, newline='') as f:
            reader = csv.DictReader(f)
            for line_number, row in enumerate(reader, start=2):
                try:
                    row['amount'] = float(row['amount'])
                    rows.append(row)
                except (ValueError, KeyError) as e:
                    print(f'Skipping malformed row {line_number}: {e}')
    except FileNotFoundError:
        print(f'{path} not found')
    return rows
```

Code Line	Explanation
Nested try inside the loop	Each individual row is validated and converted independently; a single malformed row is skipped and logged without aborting the processing of every other, valid row in the file, following the nested-handling guidance from Part 18.2.
enumerate(reader, start=2)	Tracks the actual line number in the source file (starting at 2, since line 1 is the header row), so the logged warning tells you exactly which line in the original file needs attention.

19.6 Reading JSON Files

```
import json

def load_json_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        raise InvalidConfigurationError(f'Config file not found: {path}') from None
    except json.JSONDecodeError as e:
        raise InvalidConfigurationError(
            f'Config file {path} contains invalid JSON at line {e.lineno}: {e.msg}'
        ) from e
```

Code Line	Explanation
json.JSONDecodeError	Raised when the file's content is not syntactically valid JSON at all; it is itself a subclass of ValueError, and carries useful structured attributes like .lineno and .msg pinpointing exactly where the malformed JSON begins.
Overall pattern	Both distinct failure modes (missing file, invalid JSON content) are translated into one single, consistent InvalidConfigurationError, giving every caller of load_json_config() one predictable exception type to catch, rather than needing to know about multiple different low-level exception types.

19.7 A Complete, Realistic Strategy

- Catch FileNotFoundError and PermissionError specifically when a clear, actionable message or fallback is possible.
- Catch format-specific parsing exceptions (json.JSONDecodeError, csv module issues, UnicodeDecodeError) close to where parsing happens, since that is where the most specific diagnostic context is available.
- Validate individual records inside a loop independently, so one bad row or entry does not abort processing of an entire otherwise-valid file.
- Consider translating low-level file and parsing exceptions into a single, domain-specific custom exception (Part 10) at a clear boundary, so the rest of the application deals with one predictable exception type.
- Always release file handles reliably, using with (Part 23) rather than manual open()/close() calls, so a failure partway through never leaves a file handle open.

20. Exceptions in Data Science

20.1 File Loading

```
import pandas as pd

def load_dataset(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        raise FileNotFoundError(f'Dataset not found: {path}') from None
    except pd.errors.EmptyDataError:
        raise ValueError(f'Dataset at {path} is empty') from None
    except pd.errors.ParserError as e:
        raise ValueError(f'Dataset at {path} could not be parsed: {e}') from e
```

Code Line	Explanation
<code>pd.errors.EmptyDataError</code>	Raised by pandas when the target file contains no data or no columns at all to parse, a common issue with automated data exports that occasionally produce empty files.
<code>pd.errors.ParserError</code>	Raised when the file's structure does not match what a CSV parser expects, for example an inconsistent number of columns across rows, often from a corrupted export or a file that is not actually CSV-formatted despite its extension.

20.2 Missing Columns

```
def validate_columns(df, required_columns):
    missing = set(required_columns) - set(df.columns)
    if missing:
        raise DataValidationError(
            f'Missing required columns: {sorted(missing)}'
        )
```

```
validate_columns(df, ['customer_id', 'amount', 'date'])
```

Using set difference (Part on Operators covers this operator behavior in depth) to compare required columns against the DataFrame's actual columns is a fast, precise, and easily testable way to validate structure before any downstream processing assumes those columns exist, preventing a much more confusing `KeyError` deep inside a later calculation.

20.3 Invalid Data Types

```
def clean_numeric_column(df, column):
    try:
        df[column] = pd.to_numeric(df[column], errors='raise')
    except ValueError as e:
        bad_rows = df[pd.to_numeric(df[column], errors='coerce').isna() &
df[column].notna()]
        raise DataValidationError(
```

```

        f'Column {column} contains {len(bad_rows)} non-numeric values, e.g.
        {bad_rows[column].iloc[0]!r}'
    ) from e

```

errors='raise' (pandas' default) surfaces a genuine ValueError the moment any value in the column cannot be converted; catching it and then re-running the conversion with errors='coerce' specifically to identify and report which rows are actually problematic gives a far more actionable error message than pandas' own default one, which does not point to the specific offending value.

20.4 Numerical Calculations With NumPy

```

import numpy as np

def normalize(values):
    values = np.asarray(values, dtype=float)
    std = values.std()
    if std == 0:
        raise ValueError('Cannot normalize: all values are identical (zero
variance)')
    return (values - values.mean()) / std

```

Code Line	Explanation
np.asarray(values, dtype=float)	Converts input into a numpy array, raising a ValueError automatically if the input contains values that cannot be converted to float at all, a first line of defense before the numeric logic even begins.
Explicit std == 0 check	NumPy does not raise an exception for floating-point division by zero the way plain Python does; it instead produces a RuntimeWarning and fills the result with inf or nan values silently. Explicitly checking for a zero standard deviation and raising a clear ValueError first prevents this class of silent numerical corruption from ever reaching the rest of the pipeline.
Data Science Specific Behavior: NumPy's default behavior for invalid floating-point operations (like division by zero or a NaN result) is to issue a warning and produce inf/nan, not to raise an exception. Code that depends on catching numerical problems must check explicitly, as shown above, or configure NumPy's error handling with np.seterr(all='raise') to convert these warnings into genuine exceptions.	

20.5 Data Validation Pipeline

```

def validate_dataset(df):
    errors = []
    required = ['customer_id', 'amount', 'signup_date']
    missing = set(required) - set(df.columns)
    if missing:
        errors.append(f'Missing columns: {sorted(missing)}')
    if 'amount' in df.columns and (df['amount'] < 0).any():
        errors.append('Column amount contains negative values')
    if df.duplicated(subset=['customer_id']).any():
        errors.append('Duplicate customer_id values found')
    if errors:
        raise DataValidationError('; '.join(errors))

```

```
return True
```

Rather than raising immediately on the first problem found, this pattern accumulates every validation failure into a single list and raises one comprehensive `DataValidationError` at the end, so a data engineer sees the complete picture of everything wrong with a dataset in one pass, instead of fixing one issue only to discover another on the next run.

20.6 Machine Learning Preprocessing

```
def encode_categorical(df, column, known_categories):
    unseen = set(df[column].unique()) - set(known_categories)
    if unseen:
        raise DataValidationError(
            f'Column {column} contains unseen categories not in training data:
{unseen}'
        )
    return df[column].map({cat: i for i, cat in enumerate(known_categories)})
```

A very real and common production issue: categorical values seen at prediction time that were never seen during training (a new product category, a new country code) will silently produce NaN or an incorrect encoding if not explicitly checked, corrupting downstream model input; validating this boundary explicitly and raising a clear, specific exception surfaces the problem immediately rather than allowing a model to silently make a nonsensical prediction on malformed input.

21. Exceptions in Machine Learning and AI

21.1 A ModelTrainer Class

```
class ModelNotTrainedError(Exception):
    pass

class ModelTrainer:
    def __init__(self, model):
        self.model = model
        self.is_trained = False

    def train(self, X, y):
        if len(X) == 0:
            raise ValueError('Cannot train on an empty dataset')
        if len(X) != len(y):
            raise ValueError(f'Feature/label length mismatch: {len(X)} vs {len(y)}')
        try:
            self.model.fit(X, y)
            self.is_trained = True
        except Exception as e:
            raise RuntimeError(f'Model training failed: {e}') from e

    def predict(self, X):
        if not self.is_trained:
            raise ModelNotTrainedError('Call train() before predict()')
        return self.model.predict(X)
```

Code Line	Explanation
Validation before training	Checking for an empty dataset and a feature/label length mismatch before ever calling the underlying model's own fit() method catches the two most common, easily preventable training failures with a clear, specific message, rather than letting an opaque internal library error surface instead.
try/except around model.fit()	Wraps the underlying ML library's own training call, translating whatever internal exception it might raise into a consistent, application-level RuntimeError, so callers of ModelTrainer do not need to know the specific exception types raised by whatever underlying library (scikit-learn, PyTorch, and so on) is being used.
ModelNotTrainedError in predict()	A custom exception (Part 10) enforcing a genuine sequencing invariant, that a model must be trained before it can be used to predict, catching a common and easy mistake early with a clear, specific message rather than an obscure internal error from the untrained model object.

21.2 A Predictor Class With Input Validation

```
class InvalidFeatureError(Exception):
    pass
```

```

class Predictor:
    def __init__(self, model, expected_features):
        self.model = model
        self.expected_features = expected_features

    def predict(self, features: dict):
        missing = set(self.expected_features) - set(features.keys())
        if missing:
            raise InvalidFeatureError(f'Missing required features:
{sorted(missing)}')
        extra = set(features.keys()) - set(self.expected_features)
        if extra:
            raise InvalidFeatureError(f'Unexpected features provided:
{sorted(extra)}')
        ordered_values = [features[f] for f in self.expected_features]
        try:
            return self.model.predict([ordered_values])[0]
        except ValueError as e:
            raise InvalidFeatureError(f'Invalid feature values: {e}') from e

```

This class defends the model against exactly the kind of malformed input that is extremely common in real inference services: a caller (often a separate web service, per Part 22) omitting a required feature, sending an unexpected extra field, or sending a value of the wrong type or range that only the underlying model's own internals would otherwise catch, with a far less clear error message than `InvalidFeatureError` provides here.

21.3 Missing Model Files

```

import pickle

def load_model(path):
    try:
        with open(path, 'rb') as f:
            return pickle.load(f)
    except FileNotFoundError:
        raise FileNotFoundError(
            f'Model file not found at {path}. Has the model been trained and saved?'
        ) from None
    except (pickle.UnpicklingError, EOFError) as e:
        raise RuntimeError(f'Model file at {path} is corrupted or incompatible:
{e}') from e

```

Distinguishing 'the file is missing' from 'the file exists but is corrupted or was saved with an incompatible library version' with two separate except clauses gives operators of a production ML system a much clearer, more actionable signal about which specific problem they need to investigate.

21.4 A DataProcessor With Pipeline-Style Error Handling

```

class PipelineStageError(Exception):
    def __init__(self, stage, original_error):
        self.stage = stage
        self.original_error = original_error
        super().__init__(f'Pipeline failed at stage '{stage}': {original_error}')

```

```

class DataProcessor:
    def __init__(self, stages):
        self.stages = stages

    def run(self, data):
        for name, stage_fn in self.stages:
            try:
                data = stage_fn(data)
            except Exception as e:
                raise PipelineStageError(name, e) from e
        return data

processor = DataProcessor([
    ('clean', clean_data),
    ('normalize', normalize_data),
    ('encode', encode_features),
])
result = processor.run(raw_data)

```

Code Line	Explanation
PipelineStageError with a stage attribute	Whatever underlying error a specific pipeline stage raises, it is wrapped in a consistent exception type that always identifies exactly which named stage failed, which is invaluable when a pipeline has many stages and a failure needs to be traced to a specific one quickly.
Generic except Exception inside run()	Justified here (unlike the general caution against broad except clauses in Part 5.7 and Part 26) because this is precisely the kind of top-level orchestration boundary described in section 5.7, where the specific stages are arbitrary, user-supplied functions whose possible failure modes cannot all be anticipated in advance.

21.5 Model Inference API Failures

```

class ModelPipeline:
    def __init__(self, preprocessor, model, postprocessor):
        self.preprocessor = preprocessor
        self.model = model
        self.postprocessor = postprocessor

    def predict(self, raw_input):
        try:
            features = self.preprocessor.transform(raw_input)
        except Exception as e:
            raise RuntimeError(f'Preprocessing failed: {e}') from e
        try:
            raw_prediction = self.model.predict(features)
        except Exception as e:
            raise RuntimeError(f'Model inference failed: {e}') from e
        try:
            return self.postprocessor.transform(raw_prediction)
        except Exception as e:

```

```
raise RuntimeError(f'Postprocessing failed: {e}') from e
```

Wrapping each of the three distinct stages, preprocessing, inference, and postprocessing, in its own separate try/except immediately identifies which stage failed without needing to inspect the traceback in detail, directly applying the narrow-try-block principle from Part 4.5 to a realistic three-stage ML inference pipeline.

21.6 Configuration Errors

```
class InvalidConfigurationError(Exception):
    pass

def build_model(config):
    required_keys = {'model_type', 'learning_rate', 'num_epochs'}
    missing = required_keys - config.keys()
    if missing:
        raise InvalidConfigurationError(f'Missing config keys: {sorted(missing)}')
    if config['learning_rate'] <= 0:
        raise InvalidConfigurationError('learning_rate must be positive')
    if config['model_type'] not in ('linear', 'tree', 'neural_net'):
        raise InvalidConfigurationError(f"Unknown model_type:
{config['model_type']}")
```

Validating an entire training configuration thoroughly before any expensive training process begins (which might run for hours) is a critical fail-fast pattern (formally introduced in Part 29) in ML engineering: discovering a typo in a configuration file after eight hours of training is far more costly than discovering it in the first second.

22. Exceptions in APIs and Web Applications

22.1 The requests Exception Family

Code Line	Explanation
<code>requests.exceptions.ConnectionError</code>	Raised when a network connection to the target server cannot be established at all: DNS failure, refused connection, or no network route.
<code>requests.exceptions.Timeout</code>	Raised when a request takes longer than the configured timeout to complete, covering both <code>ConnectTimeout</code> (establishing the connection itself timed out) and <code>ReadTimeout</code> (the connection was made but the server did not respond with data in time).
<code>requests.exceptions.HTTPError</code>	Raised (only when explicitly triggered by calling <code>response.raise_for_status()</code>) when the server responds successfully at the network level but with an HTTP error status code, like 404 or 500.
<code>requests.exceptions.RequestException</code>	The base class for essentially every exception the requests library raises; catching this catches any of the above at once.

22.2 A Professional API Client Pattern

```
import requests

class APIClientError(Exception):
    pass

def fetch_user(user_id, timeout=5):
    try:
        response = requests.get(f'https://api.example.com/users/{user_id}',
                                timeout=timeout)
        response.raise_for_status()
        return response.json()
    except requests.exceptions.Timeout as e:
        raise APIClientError(f'Request for user {user_id} timed out') from e
    except requests.exceptions.ConnectionError as e:
        raise APIClientError('Could not connect to the API server') from e
    except requests.exceptions.HTTPError as e:
        status = e.response.status_code
        if status == 404:
            raise APIClientError(f'User {user_id} not found') from e
        elif status == 401:
            raise APIClientError('Authentication failed; check API credentials')
        from e
        elif status == 429:
            raise APIClientError('Rate limit exceeded; slow down requests') from e
            raise APIClientError(f'API returned an unexpected error: {status}') from e
    except requests.exceptions.JSONDecodeError as e:
        raise APIClientError('API returned a response that was not valid JSON') from e
```

Code Line	Explanation
<code>timeout=timeout</code> passed to <code>requests.get()</code>	Always specifying an explicit timeout is essential in production code; without one, a hung network request could block the calling code indefinitely, with no exception ever raised at all.
<code>response.raise_for_status()</code>	Converts an HTTP error status code (4xx or 5xx) into an actual raised <code>HTTPError</code> exception; without calling this, requests does not raise an exception for error status codes on its own, and a 404 or 500 response would otherwise be silently treated as a normal, successful response.
Branching on <code>e.response.status_code</code>	Distinguishes different categories of HTTP failure (not found, authentication, rate limiting) that each warrant a genuinely different message or response strategy, rather than treating every HTTP error identically.
Every branch raises the same <code>APIClientError</code> type	Gives every caller of <code>fetch_user()</code> a single, consistent, predictable exception type to catch, regardless of which specific underlying network or HTTP problem actually occurred, following the exception translation pattern from Part 13.6 and Part 29.

22.3 JSON Decoding Errors

```
import json

def parse_api_response(response_text):
    try:
        return json.loads(response_text)
    except json.JSONDecodeError as e:
        raise APIClientError(
            f'API returned malformed JSON at position {e.pos}: {e.msg}'
        ) from e
```

An API that is expected to always return JSON occasionally does not, particularly during an outage (returning an HTML error page instead) or a misconfiguration; explicitly catching `json.JSONDecodeError` here prevents that situation from surfacing as a confusing, generic exception deep inside whatever code consumes the parsed response.

22.4 Authentication Errors and Rate Limits

```
class AuthenticationError(APIClientError):
    pass

class RateLimitError(APIClientError):
    def __init__(self, retry_after):
        self.retry_after = retry_after
        super().__init__(f'Rate limited; retry after {retry_after} seconds')
```

Giving authentication failures and rate limit errors their own specific subclasses of a common `APIClientError` base (Part 10.4) lets calling code respond very differently to each: an `AuthenticationError` likely means credentials need to be refreshed or the request aborted entirely, while a `RateLimitError`, especially one carrying a structured `retry_after` attribute, is a clear, direct signal to pause and retry later, connecting directly to the retry logic covered fully in Part 30.

22.5 A Complete Web Request Handler Pattern

```
def handle_get_user(request, user_id):
    try:
        user = fetch_user(user_id)
        return {'status': 200, 'body': user}
    except APIClientError as e:
        logger.warning(f'API client error handling request for user {user_id}: {e}')
        return {'status': 502, 'body': {'error': 'Upstream service unavailable'}}
    except Exception as e:
        logger.exception(f'Unexpected error handling request for user {user_id}')
        return {'status': 500, 'body': {'error': 'Internal server error'}}
```

This illustrates a critical production pattern: a request handler at the boundary of a web application should never let an exception propagate all the way out uncaught (which would typically crash the request, or in some frameworks the entire worker process); it should catch anticipated failures specifically (`APIClientError`, mapped to an appropriate HTTP status), catch any remaining unexpected exception broadly as a last resort (per the justified `except Exception` usage from Part 5.7), log full details for debugging internally, and always return a clean, safe response rather than ever leaking an internal traceback or sensitive details to an external caller (revisited in depth in Part 25.9 on sensitive information in logs).

23. Exceptions and Context Managers

23.1 The with Statement and Guaranteed Cleanup

```
with open('data.txt') as f:
    content = f.read()
print('File is now closed, even if read() had raised an exception')
```

The with statement is, in essence, syntax that automates exactly the try/finally cleanup pattern from Part 7, but more concisely and reliably. Whatever object follows with (here, the file object returned by open()) is called a context manager, and it guarantees that its own cleanup logic runs when the with block ends, whether that end comes from normal completion or from an exception propagating through the block.

23.2 `__enter__` and `__exit__`

```
class ManagedResource:
    def __enter__(self):
        print('Acquiring resource')
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print('Releasing resource')
        return False

with ManagedResource() as resource:
    print('Using resource')
    raise ValueError('Something went wrong')
```

Code Line	Explanation
<code>__enter__(self)</code>	Called automatically at the very start of the with block; whatever it returns is bound to the variable after as (here, resource). This is where setup or acquisition logic belongs.
<code>__exit__(self, exc_type, exc_value, traceback)</code>	Called automatically when the with block ends, for any reason at all, including an exception; Python always calls it, exactly mirroring the guarantee finally provides.
The three <code>exc_</code> parameters	If the with block completed normally, all three are None. If an exception occurred inside the block, they are populated with the exception's type, the exception instance itself, and its traceback object, exactly the same three pieces of information <code>sys.exc_info()</code> returns (Part 12.5).
The return value of <code>__exit__</code>	This is critically important: returning False (or anything falsy, including simply not returning at all) tells Python to let the exception continue propagating normally after <code>__exit__</code> finishes; returning True tells Python to suppress the exception entirely, as if it had been caught and handled.

Running the example above prints 'Acquiring resource', then 'Using resource', then 'Releasing resource' (proving `__exit__` ran despite the exception), and then the `ValueError` still propagates normally afterward, since `__exit__` returned False.

23.3 Suppressing Exceptions in `__exit__`

```
class IgnoreErrors:
    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is not None:
            print(f'Suppressed an exception: {exc_type.__name__}: {exc_value}')
            return True

with IgnoreErrors():
    raise ValueError('This will be suppressed')
print('Execution continues normally here')
```

Returning True from `__exit__` deliberately swallows the exception, and execution continues immediately after the with block as if nothing had gone wrong. This is a powerful capability, and precisely because it is powerful, it carries the same risk as any other exception-suppressing pattern discussed in Part 26: it should only be done deliberately and narrowly, never as a way to broadly and silently hide failures.

23.4 Writing a Custom Context Manager for Real Cleanup

```
class DatabaseConnection:
    def __init__(self, dsn):
        self.dsn = dsn
        self.connection = None

    def __enter__(self):
        self.connection = connect(self.dsn)
        return self.connection

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is not None:
            self.connection.rollback()
        else:
            self.connection.commit()
        self.connection.close()
        return False

with DatabaseConnection(dsn) as conn:
    conn.execute('UPDATE accounts SET balance = balance - 100 WHERE id = 1')
```

This directly reimplements the exact commit-on-success, rollback-on-failure, always-close pattern first introduced with try/except/finally in Part 7.6, but packaged as a reusable class; any code that needs this same database connection behavior can now simply write `with DatabaseConnection(dsn) as conn:`, without repeating the try/except/finally logic every single time.

23.5 The `contextlib` Shortcut

```
from contextlib import contextmanager

@contextmanager
```

```
def database_connection(dsn):
    conn = connect(dsn)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

Code Line	Explanation
@contextmanager	A decorator from the standard library's contextlib module that turns an ordinary generator function into a full context manager, without needing to manually write a class with separate <code>__enter__</code> and <code>__exit__</code> methods.
yield conn	Everything before yield acts as <code>__enter__</code> (setup, here establishing the connection); the yielded value becomes what is bound after as; everything after yield acts as <code>__exit__</code> (cleanup).
try/except/finally around the yield	If the code inside the with block (which resumes right where yield is) raises an exception, it is raised at the point of yield inside this generator, allowing this same try/except/finally structure to catch it, roll back, and still guarantee the connection is closed, precisely mirroring the class-based version above with noticeably less code.
Design Guidance: Whenever an object needs guaranteed setup and teardown logic, especially teardown that must run even when an exception occurs, prefer a context manager (either a class with <code>__enter__</code> / <code>__exit__</code> or a <code>@contextmanager</code> generator) over a manual try/finally repeated at every call site. It packages the guarantee once, correctly, in a single reusable place.	

24. Exceptions and Classes/OOP

24.1 Custom Exceptions Recap in an OOP Context

Part 10 already covered how to define custom exception classes in full detail. This part focuses specifically on how exceptions interact with the rest of a class's design: methods that raise, exception hierarchies that mirror a class hierarchy, and how to design classes whose failure behavior is clear and predictable for whoever uses them.

24.2 Exceptions Inside Methods

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('Deposit amount must be positive')
        self._balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError('Withdrawal amount must be positive')
        if amount > self._balance:
            raise InsufficientFundsError(self._balance, amount)
        self._balance -= amount
```

Methods raise exceptions in exactly the same way as ordinary functions (Part 9); the only real distinction is that a method typically validates against the object's own internal state (`self._balance` here), rather than only its arguments, since that state represents the object's specific, current invariants.

24.3 Exception Handling Inside Methods

```
class ConfigLoader:
    def __init__(self, path):
        self.path = path
        self._config = None

    def load(self):
        try:
            with open(self.path) as f:
                self._config = json.load(f)
        except FileNotFoundError:
            self._config = {}
            print(f'No config found at {self.path}; using defaults')
```

A method can just as legitimately catch and fully handle its own exceptions internally (Part 16.2), providing a graceful fallback that keeps the object in a valid, usable state (an empty config dictionary here) rather than forcing every caller of `load()` to handle a `FileNotFoundError` themselves.

24.4 Domain-Specific Exception Hierarchies Tied to a Class Hierarchy

```
class ShapeError(Exception):
    pass

class InvalidDimensionError(ShapeError):
    pass

class Shape:
    def area(self):
        raise NotImplementedError('Subclasses must implement area()')

class Rectangle(Shape):
    def __init__(self, width, height):
        if width <= 0 or height <= 0:
            raise InvalidDimensionError(f'Width and height must be positive, got {width}, {height}')
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

Code Line	Explanation
NotImplementedError in the base Shape.area()	A now-familiar pattern from Part 3.8.2: signals that Shape is meant to act as an abstract base, and any subclass that forgets to override area() will fail clearly and immediately, rather than silently returning something meaningless.
InvalidDimensionError(ShapeError)	A domain exception hierarchy (Part 10.4) built specifically around this class family, letting calling code catch ShapeError broadly to handle any problem with any Shape subclass uniformly, or InvalidDimensionError specifically for this one narrower situation.

24.5 Designing Robust Classes: Principles

- Validate constructor arguments immediately in `__init__`, so an object can never exist in an invalid state in the first place (as Rectangle does above).
- Raise specific, well-named exceptions from methods that detect a violated invariant, rather than allowing an operation to proceed and fail confusingly later.
- Decide deliberately, method by method, whether a given failure should be caught and handled internally (providing a fallback) or allowed to propagate to the caller (when the object genuinely cannot make that decision itself), following the same reasoning from Part 16.3.
- Where a class represents a specific domain (banking, shapes, ML pipelines), define a small, purposeful exception hierarchy specific to that domain, rather than raising generic built-in exceptions for every domain-specific failure.
- Document, through docstrings or type hints, which exceptions a public method can raise, so users of the class know what to anticipate and catch without needing to read its full implementation.

25. Logging and Exceptions

25.1 Why print() Is Not Enough

- `print()` output cannot be easily filtered by severity; there is no built-in way to show only warnings and errors while suppressing routine informational messages.
- `print()` output typically goes only to the console and is lost once a terminal session ends, with no persistent record for later investigation.
- `print()` provides no built-in timestamp, module name, or severity level, all of which are essential context when investigating a problem in a production system after the fact.
- Production systems need to route messages to different destinations at once (a file, a monitoring service, the console) and the logging module supports this natively, while `print()` does not.

25.2 Python's logging Module: Basic Setup

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(name)s: %(message)s'
)

logger = logging.getLogger(__name__)
```

Code Line	Explanation
<code>logging.basicConfig(level=logging.INFO, ...)</code>	Configures the root logging setup once, typically near the start of an application; <code>level=logging.INFO</code> means messages at INFO severity and above are shown, while lower-severity DEBUG messages are suppressed by default.
<code>format=...</code>	A template string controlling exactly what each log line contains; <code>%(asctime)s</code> inserts a timestamp, <code>%(levelname)s</code> inserts the severity (INFO, WARNING, and so on), <code>%(name)s</code> inserts the logger's name, and <code>%(message)s</code> inserts the actual log message.
<code>logging.getLogger(__name__)</code>	Creates (or retrieves) a logger scoped to the current module's name; using <code>__name__</code> this way, rather than a hardcoded string, is the standard, idiomatic convention, since it automatically produces a distinct, correctly named logger in every module that uses this same pattern.

25.3 Log Levels

Code Line	Explanation
DEBUG	Detailed diagnostic information, typically useful only during active development or deep troubleshooting; usually suppressed in normal production operation.
INFO	Confirmation that things are working as expected; routine operational messages (a request was processed, a job started).
WARNING	Something unexpected happened, or a potential future problem was noted, but the program is still working as intended.

Code Line	Explanation
ERROR	A more serious problem: the program was unable to perform a specific operation or function correctly.
CRITICAL	A very serious error, suggesting the program itself may be unable to continue running at all.

25.4 Logging Exceptions With `logger.exception()`

```
def process_order(order_id):
    try:
        charge_payment(order_id)
    except PaymentError:
        logger.exception(f'Payment failed for order {order_id}')
        raise
```

Code Line	Explanation
<code>logger.exception(message)</code>	A convenience method that must be called from inside an except block; it logs the given message at ERROR level and automatically includes the full traceback of the exception currently being handled, without needing to format or capture it manually.
Equivalent longer form	<code>logger.error(message, exc_info=True)</code> does exactly the same thing; <code>logger.exception()</code> is simply shorthand specifically for the common case of logging from within an except block.

The traceback captured in the log is exactly as valuable during a later production investigation as it is when printed directly to a terminal during development; `logger.exception()` ensures that same rich diagnostic detail is preserved and searchable in whatever logging destination the application is configured to use, rather than being lost the moment the terminal session ends.

25.5 Tracebacks in Logs: A Worked Example

```
2026-03-14 09:12:03,481 ERROR app.orders: Payment failed for order 5821
Traceback (most recent call last):
  File "orders.py", line 22, in process_order
    charge_payment(order_id)
  File "payments.py", line 8, in charge_payment
    raise PaymentError('Card declined')
PaymentError: Card declined
```

This is what a typical production log entry produced by `logger.exception()` looks like: the formatted log line (timestamp, level, logger name, message) followed by the complete traceback, giving anyone reviewing the logs later everything needed to diagnose the failure, exactly the same information a developer would have seen locally at the moment it happened.

25.6 What Information Should Be Logged

- The exception type and message (captured automatically by `logger.exception()`).
- Enough surrounding business context to understand what operation was being attempted: an order ID, a user ID, a request ID, a file path.

- The full traceback, for any error-level or higher event, to support later debugging without needing to reproduce the issue.
- A consistent, structured format (via `format=` in `basicConfig`, or structured/JSON logging in larger systems) that supports later searching and filtering across a large volume of logs.

25.7 What Sensitive Information Should NOT Be Logged

- Passwords, API keys, authentication tokens, or session identifiers, even in an error message or exception's arguments.
- Full credit card numbers or other complete financial account details; log only a masked or truncated reference if any identifier is needed at all.
- Personally identifiable information beyond what is genuinely necessary for debugging (a user ID is usually sufficient; a full name, home address, or date of birth usually is not).
- Raw request or response bodies from an API without first scrubbing them, since these frequently contain exactly the sensitive fields listed above embedded within them.

```
# WRONG: logs the raw exception, which might contain the user's password
except AuthenticationError as e:
    logger.exception(f'Login failed: {e}')

# RIGHT: logs only what is needed to diagnose the problem
except AuthenticationError:
    logger.warning(f'Login failed for username={username} (invalid credentials)')
```

Security Warning: Before logging an exception's message directly, consider what data could realistically have flowed into it. An exception raised during authentication or payment processing can easily carry sensitive values inside its own message or attributes; log deliberately, not by default, in code paths that touch sensitive data.

26. Good vs Bad Exception Handling

26.1 Bad Practice: Bare except

```
try:
    do_something()
except:
    pass
```

Covered fully in Part 5.8: catches everything including KeyboardInterrupt and SystemExit, and here combined with pass, it also completely discards all information about what went wrong. This is close to the single worst pattern possible in exception handling.

26.2 Bad Practice: Catching Everything Indiscriminately

```
try:
    result = complex_calculation(data)
except Exception:
    result = 0
```

Even though this at least excludes KeyboardInterrupt and SystemExit, it still catches every conceivable application error, a TypeError from a genuine bug, a ValueError from bad data, a MemoryError from resource exhaustion, and silently substitutes 0 for all of them alike, with no way to distinguish a truly recoverable situation from a serious bug that urgently needs fixing.

26.3 Bad Practice: Silently Ignoring Errors

```
try:
    save_to_database(record)
except Exception:
    pass
```

This is arguably worse than letting the program crash outright: the calling code proceeds as if save_to_database() succeeded, when in fact the data was silently never saved at all, and there is no log, no message, no trace of the failure anywhere. Data loss of exactly this kind, caused by a swallowed exception, is one of the most damaging and hardest-to-diagnose classes of production bug.

26.4 Bad Practice: Using Exceptions for Normal Control Flow

```
# Bad: using an exception to check for a routine, expected condition
try:
    value = my_dict[key]
    exists = True
except KeyError:
    exists = False

# Better: use the tool built specifically for this
exists = key in my_dict
```

Exceptions exist to signal genuinely exceptional, unusual conditions; using try/except to implement what is really an ordinary, routine check makes code harder to read and is typically slower than the direct, purpose-built

alternative (in this case, the `in` operator). This does not mean exceptions should never be used for control flow at all, EAFP-style code (Easier to Ask Forgiveness than Permission) is a legitimate and common Python idiom, but it should be reserved for cases where the exceptional path is genuinely uncommon, not for routine checks that have a simple, direct alternative.

26.5 Bad Practice: Giant try Blocks

Covered fully with a worked example in Part 4.5: wrapping many unrelated operations in one enormous try block makes it impossible to tell which operation actually failed and prevents responding differently to different failure types.

26.6 Bad Practice: Hiding Errors Behind Vague Messages

```
except Exception as e:  
    print('Error occurred')
```

The actual exception object `e` is completely discarded here; the printed message provides no information whatsoever about what went wrong, where, or why, forcing anyone investigating the problem to reproduce it from scratch rather than simply reading what already happened.

26.7 Good Practice: Catch Specific Exceptions

```
try:  
    age = int(user_input)  
except ValueError:  
    print(f'{user_input!r} is not a valid whole number')
```

Catches exactly the anticipated failure mode, with a message specific enough to be genuinely useful, and lets any other, unanticipated exception type propagate visibly rather than being silently absorbed.

26.8 Good Practice: Keep try Blocks Focused

Covered fully in Part 4; each try block should wrap only the smallest reasonable scope of code that can actually raise the specific exception(s) being handled.

26.9 Good Practice: Preserve Useful Error Information

```
except ValueError as e:  
    raise DataValidationError(f'Invalid record: {e}') from e
```

Uses exception chaining (Part 13) to translate a low-level exception into a more meaningful one, without ever discarding the original error's details.

26.10 Good Practice: Use Custom Exceptions Where Appropriate

Covered fully in Part 10: a well-designed custom exception hierarchy makes calling code's handling logic clearer and more precise than relying only on generic built-in exceptions for domain-specific failures.

26.11 Good Practice: Log Appropriately, Clean Up Resources, Fail Clearly

```
def process_payment(order):  
    try:  
        charge(order)  
    except PaymentDeclinedError as e:  
        logger.warning(f'Payment declined for order {order.id}: {e}')  
        raise  
    except PaymentGatewayError as e:  
        logger.exception(f'Payment gateway error for order {order.id}')  
        raise
```

This combines several good practices at once from across this handbook: specific exception types, appropriate log severity for each (a decline is a routine WARNING, a gateway failure is a genuine ERROR worth full traceback capture with `logger.exception()`), and a clean re-raise (Part 14) so the failure remains visible to whatever called `process_payment()`, rather than being silently absorbed at this level.

27. Exception Handling Best Practices

27.1 What to Catch

- Catch the most specific exception type that accurately describes the failure you are prepared to handle.
- Catch a family via a shared parent class (like `LookupError` or a custom domain base class) when the same handling genuinely applies to every member of that family.
- Use `except Exception` only at genuine top-level boundaries (a request handler, a plugin runner, a background job loop) where truly arbitrary failures must not crash the whole system.

27.2 What Not to Catch

- Do not use a bare `except`: (Part 5.8).
- Do not catch exceptions you have no real plan for; if there is nothing meaningful to do in response, let the exception propagate instead.
- Do not catch broadly and then silently discard the exception (Part 26.3).

27.3 When to Raise

- Raise as soon as an invalid state or invalid input is detected, following the fail-fast principle (Part 29.2), rather than allowing invalid data to propagate further into the system.
- Raise the most specific, accurate built-in exception type, or a custom exception when the failure has genuine domain meaning (Part 10.6).
- Always include a clear, specific message, ideally including the actual offending value.

27.4 When to Re-Raise

- Re-raise (bare `raise`) when a handler performs a genuinely useful side effect (logging, an alert, a metric) but cannot itself resolve the underlying problem (Part 14.1).
- Re-raise a new, translated exception (with `from e`) when the original exception type is too low-level for the caller's needs (Part 13.6).

27.5 When to Use Custom Exceptions

Revisit the criteria from Part 10.6: a specific, named domain concept, a genuine need for callers to catch it separately, structured data beyond a text message, or a public library boundary that benefits from a stable, meaningful exception type.

27.6 Logging

- Use the logging module, never `print()`, for anything beyond quick local debugging (Part 25.1).
- Use `logger.exception()` inside `except` blocks to automatically capture the full traceback.
- Choose an appropriate severity level for each situation, not `ERROR` by default for everything (Part 25.3).
- Never log sensitive information (Part 25.7).

27.7 User-Facing Messages

- Internal exception details (a raw traceback, an internal exception type name, an internal file path) generally should not be shown directly to an end user; translate them into a clear, non-technical message instead.
- Log the full technical detail internally, while showing the user only what they need to know and, where relevant, what they can do about it.

27.8 Security

- Never expose stack traces, internal file paths, or internal exception details in a public-facing API response or web page; this can leak sensitive information about your system's internals to an attacker.
- Never log secrets, credentials, or complete personal data (Part 25.7).

27.9 Maintainability and Debugging

- Keep try blocks narrow and purposeful (Part 4).
- Prefer specific exception types over broad ones, both when catching and when raising.
- Use exception chaining (Part 13) so translating an exception never destroys the original diagnostic trail.
- Document which exceptions a public function or method can raise.

27.10 Production Systems

- Ensure every request-handling or job-processing boundary has a final, broad safety net (Part 22.5) so an unexpected exception never crashes the entire process.
- Ensure resource cleanup (files, connections, locks) is guaranteed with finally or a context manager (Parts 7 and 23), never left to only the successful path.
- Monitor and alert on ERROR- and CRITICAL-level log events, since these represent genuine problems requiring attention.
- Build retry logic (Part 30) for genuinely transient failures, and fail clearly and permanently for non-retryable ones.

28. Debugging Exception-Related Problems

28.1 A Systematic Debugging Process

- 1. Read the traceback fully, starting from the bottom (Part 2.3).
- 2. Identify the exact exception type and message.
- 3. Locate the specific failing line, and the specific sub-expression on it if Python 3.11+ underlining is available (Part 2.2).
- 4. Inspect the actual values of the variables involved at that point, not just their expected values.
- 5. Reproduce the problem reliably, ideally with a minimal example isolating just the failing behavior.
- 6. Determine the root cause, which, as shown in Part 2.6, is not always the same line where the exception was actually raised.
- 7. Fix the underlying issue, not merely the symptom (avoid, for example, simply catching and ignoring the exception without addressing why it occurred).
- 8. Test the fix, including the specific failing scenario and any related edge cases nearby.

28.2 Debugging Exercise 1

```
def get_average_price(products):
    total = sum(p['price'] for p in products)
    return total / len(products)

print(get_average_price([]))
```

Code Line	Explanation
Step 1-2: Traceback and exception type	ZeroDivisionError: division by zero, raised on the return line.
Step 3-4: Failing line and variable inspection	len(products) is 0, since products is an empty list; total is 0 (sum of an empty generator), so the division itself is 0 / 0.
Step 5-6: Reproduction and root cause	Reliably reproduced by calling with an empty list; the root cause is that the function assumes products always has at least one element, an assumption never validated.
Step 7: Fix	Add a guard: if not products: raise ValueError('Cannot compute average of an empty product list'), or return a sentinel like None if an empty list is a legitimately expected input.

28.3 Debugging Exercise 2

```
def get_discounted_prices(prices, discounts):
    return [p * (1 - discounts[i]) for i, p in enumerate(prices)]

prices = [100, 200, 300]
discounts = [0.1, 0.2]
print(get_discounted_prices(prices, discounts))
```

Code Line	Explanation
Step 1-2	IndexError: list index out of range.
Step 3-4	The failure occurs on the third loop iteration (i=2), since discounts only has 2 elements (valid indices 0 and 1), but prices has 3.
Step 5-6	Root cause: the function silently assumes prices and discounts are always the same length, which is never checked.
Step 7	Add validation at the top: if len(prices) != len(discounts): raise ValueError(f'Length mismatch: {len(prices)} prices vs {len(discounts)} discounts').

28.4 Debugging Exercise 3 (Logical Error, No Exception at All)

```
def is_even(n):
    if n % 2 == 1:
        return True
    return False

print(is_even(4))
```

Code Line	Explanation
Why this is harder to debug	No exception is raised at all, and the code runs to completion; but is_even(4) incorrectly prints False, since the condition's logic is backward (it actually checks for odd numbers, not even ones).
Diagnostic approach	Since there is no traceback to read, this requires comparing actual output against expected output directly (per section 1.6), tracing through the logic by hand, or using a debugger to step through and inspect n % 2 at the point of the condition.
Fix	Correct the condition to n % 2 == 0, or more simply, return n % 2 == 0 directly.

29. Exception Handling Design Patterns

29.1 Validation Before Processing

```
def process_order(order):
    if order.quantity <= 0:
        raise ValueError('Quantity must be positive')
    if not order.customer_id:
        raise ValueError('Customer ID is required')
    # main processing logic only runs once every precondition holds
    charge_customer(order)
    ship_order(order)
```

Validate every precondition explicitly at the very start of an operation, before any of its real work begins, so failures are caught immediately and clearly rather than partway through a multi-step process that might then need to be undone.

29.2 Fail-Fast

A closely related principle: detect and raise on invalid state as early as possible, rather than allowing it to propagate deeper into a system where it becomes harder to trace back to its true origin. The configuration validation example in Part 21.6 (validating an entire ML training config before an hours-long training run begins) is a clear, high-stakes illustration of why failing fast is valuable, not just tidy.

29.3 Graceful Degradation

```
def get_recommendations(user_id):
    try:
        return fetch_personalized_recommendations(user_id)
    except RecommendationServiceError:
        logger.warning(f'Personalized recommendations unavailable for {user_id},
using fallback')
        return get_popular_items()
```

When a non-critical operation fails, falling back to a simpler, still-useful alternative (generic popular items instead of a personalized recommendation) rather than failing the entire request outright. This pattern is appropriate specifically when a genuinely acceptable fallback exists; it is not appropriate for critical operations like a financial transaction, where a silent, degraded 'success' could be actively harmful.

29.4 Retry Logic (Preview)

For failures that are genuinely transient (a momentary network blip, a temporarily overloaded service), retrying the operation, ideally with a deliberate delay, is often more appropriate than either failing immediately or falling back permanently. This pattern is significant enough to warrant its own complete, dedicated treatment in Part 30.

29.5 Fallback Behavior

A more general version of graceful degradation: providing a specific, alternative code path when the primary approach fails, whether that is a cached value, a default configuration, or a simpler algorithm. The distinguishing

feature of this pattern is that the fallback is a deliberate, pre-planned alternative, not merely swallowing the exception and hoping for the best.

29.6 Exception Translation

Introduced in Part 13.6 and used repeatedly throughout Parts 19 through 22: catching a low-level, implementation-specific exception and raising a higher-level, domain-meaningful one in its place, ideally chained with `from e` so the original cause remains fully traceable. This pattern is what allows the rest of an application to depend on a small, stable set of meaningful exception types, rather than needing to know about every specific exception a given underlying library might raise.

29.7 Exception Chaining as a Pattern

Beyond its mechanical definition in Part 13, exception chaining is itself a design pattern: a deliberate practice of always preserving the causal link between a new exception and whatever originally triggered it, ensuring that translating or wrapping an exception (as in the pattern above) never comes at the cost of losing the original diagnostic detail.

29.8 Centralized Error Handling

```
def handle_request(request):
    try:
        return route_request(request)
    except ValidationError as e:
        return error_response(400, str(e))
    except AuthenticationError as e:
        return error_response(401, str(e))
    except NotFoundError as e:
        return error_response(404, str(e))
    except Exception:
        logger.exception('Unhandled error processing request')
        return error_response(500, 'Internal server error')
```

Rather than scattering try/except logic throughout many individual route handlers, a single, centralized handler (at the true boundary of the system, revisiting the Part 22.5 pattern) maps every anticipated exception type to an appropriate response in one consistent place, with one final, broad safety net for anything unanticipated. This keeps individual business logic functions free to simply raise meaningful exceptions (following validation-before-processing) without needing to know anything about how those exceptions are ultimately translated into a user-facing response.

30. Retry Logic

30.1 Why Retry at All

Many failures encountered when interacting with external systems, networks, APIs, databases, are transient: momentary, self-resolving conditions rather than persistent problems. A dropped connection, a service that is briefly overloaded, or a temporary DNS hiccup will often succeed if the exact same operation is simply attempted again a moment later. Retry logic formalizes this common-sense idea into a reliable, bounded pattern.

30.2 A Basic Retry Loop

```
import time

def fetch_with_retry(url, max_retries=3, delay=2):
    for attempt in range(1, max_retries + 1):
        try:
            return requests.get(url, timeout=5)
        except requests.exceptions.ConnectionError as e:
            if attempt == max_retries:
                raise
            print(f'Attempt {attempt} failed: {e}. Retrying in {delay}s...')
            time.sleep(delay)
```

Code Line	Explanation
for attempt in range(1, max_retries + 1):	Bounds the total number of attempts explicitly; this loop will try at most max_retries times before giving up, which is essential (see section 30.5).
if attempt == max_retries: raise	On the final allowed attempt, if it still fails, the exception is allowed to propagate normally rather than being retried again, ensuring the caller is still correctly informed of the ultimate failure.
time.sleep(delay)	Pauses execution for a fixed delay before the next attempt, giving whatever transient condition caused the failure a chance to resolve itself before trying again.

30.3 Maximum Retries and Delays

Every retry mechanism needs an explicit upper bound on attempts (max_retries above); without one, a persistently failing operation (not merely a transient blip, but a genuine, ongoing outage) would retry forever, consuming resources indefinitely and never actually informing anyone that something is seriously wrong.

30.4 Exponential Backoff

```
def fetch_with_backoff(url, max_retries=5, base_delay=1):
    for attempt in range(1, max_retries + 1):
        try:
            return requests.get(url, timeout=5)
        except requests.exceptions.ConnectionError:
            if attempt == max_retries:
```

```

        raise
    delay = base_delay * (2 ** (attempt - 1))
    print(f'Attempt {attempt} failed. Retrying in {delay}s...')
    time.sleep(delay)

```

Code Line	Explanation
delay = base_delay * (2 ** (attempt - 1))	Instead of a fixed delay, the wait time doubles with each successive attempt (1s, 2s, 4s, 8s, ...); this is exponential backoff, and it is the standard, professionally expected approach for retrying against external services.
Why exponential, not fixed	A fixed short delay retried by many simultaneous callers can itself contribute to overwhelming an already-struggling service (a 'thundering herd'); a growing delay backs off more aggressively over time, giving a genuinely struggling service meaningfully more room to recover, while still retrying quickly for the common case of a truly momentary blip.

30.5 Retryable vs Non-Retryable Errors

```

def fetch_with_smart_retry(url, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()
            return response
        except requests.exceptions.HTTPError as e:
            status = e.response.status_code
            if status in (400, 401, 403, 404):
                raise # client errors: retrying will never help, fail immediately
            if attempt == max_retries:
                raise
        except requests.exceptions.ConnectionError:
            if attempt == max_retries:
                raise
    time.sleep(2 ** attempt)

```

Code Line	Explanation
Non-retryable: 400, 401, 403, 404	These represent problems with the request itself (malformed input, invalid credentials, a resource that genuinely does not exist); retrying the exact same request will produce the exact same failure every time, so retrying only wastes time and resources.
Retryable: ConnectionError, 5xx server errors, timeouts	These typically represent transient conditions on the server or network side that are genuinely likely to resolve on their own, making a retry a sensible strategy.

Critical Distinction: Blindly retrying every kind of failure is a common and costly mistake. Retrying a non-retryable error wastes time, delays a failure the caller needed to know about immediately, and in the case of a request that has a side effect (like charging a payment), can be actively dangerous if not designed carefully around idempotency.

30.6 Avoiding Infinite Retries

- Always set an explicit, finite `max_retries`; never loop `while True`: around a retry without a hard upper bound.
- Always re-raise (or otherwise clearly surface) the failure once the retry budget is exhausted, rather than silently giving up (revisiting Part 26.3).
- Consider a maximum total elapsed time budget in addition to a maximum attempt count, particularly for user-facing operations where an upper bound on total wait time matters as much as the number of attempts.

31. Real-World Projects

Each project below is a complete, runnable piece of code that draws together many of the techniques covered throughout this handbook, followed by a step-by-step explanation of the key exception-handling decisions made.

31.1 Project: Robust Student Registration System

```
class RegistrationError(Exception):
    pass

class DuplicateStudentError(RegistrationError):
    pass

class InvalidAgeError(RegistrationError):
    pass

class CourseFullError(RegistrationError):
    pass

class StudentRegistry:
    def __init__(self, course_capacity=30):
        self._students = {}
        self._capacity = course_capacity

    def register(self, student_id, name, age):
        if student_id in self._students:
            raise DuplicateStudentError(f'Student {student_id} is already
registered')
        if not isinstance(age, int) or not (5 <= age <= 100):
            raise InvalidAgeError(f'Invalid age for student {student_id}: {age}')
        if len(self._students) >= self._capacity:
            raise CourseFullError(f'Course is full ({self._capacity} students)')
        self._students[student_id] = {'name': name, 'age': age}
        return self._students[student_id]

    def register_batch(self, records):
        results = {'registered': [], 'failed': []}
        for record in records:
            try:
                self.register(record['id'], record['name'], record['age'])
                results['registered'].append(record['id'])
            except RegistrationError as e:
                results['failed'].append({'id': record['id'], 'reason': str(e)})
        return results
```

Code Line	Explanation
A small, purposeful exception hierarchy	RegistrationError as a base, with three specific subclasses, following the Part 10.4 pattern; register_batch() can catch RegistrationError once and correctly handle any of the three specific failures uniformly.
register_batch() never lets	Directly applying the nested-handling guidance from Part 18: each

Code Line	Explanation
one bad record stop the whole batch	record is attempted independently, and a failure is recorded rather than propagated, so a single invalid record in a batch of 200 does not prevent the other 199 from registering.
Why the individual register() method still raises rather than returning a status	This keeps register() itself simple and reusable for both single-record and batch use; register_batch() is the layer that decides how to aggregate failures, following the propagation and function-design reasoning from Parts 16 and 17.

31.2 Project: Banking Transaction System

```
import logging
logger = logging.getLogger('banking')

class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance, self.amount = balance, amount
        super().__init__(f'Cannot withdraw {amount}: balance is {balance}')

class Account:
    def __init__(self, account_id, balance=0):
        self.account_id = account_id
        self.balance = balance

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError('Withdrawal amount must be positive')
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('Deposit amount must be positive')
        self.balance += amount

def transfer(from_account, to_account, amount):
    try:
        from_account.withdraw(amount)
    except InsufficientFundsError:
        logger.warning(f'Transfer failed: {from_account.account_id} has insufficient funds')
        raise
    try:
        to_account.deposit(amount)
    except Exception:
        from_account.deposit(amount) # roll back the withdrawal
        logger.exception(f'Deposit to {to_account.account_id} failed; withdrawal rolled back')
        raise
```

```
logger.info(f'Transferred {amount} from {from_account.account_id} to
{to_account.account_id}')
```

Code Line	Explanation
Manual rollback in transfer()	Since two independent objects are involved (not a single database transaction), a failure in the second step (deposit) must manually reverse the first step (withdraw) to avoid leaving money in an inconsistent state, half-transferred; this hand-written rollback mirrors what a real database transaction's automatic rollback (Part 7.6) provides.
Appropriate log severities	A routine insufficient-funds failure logs at WARNING (an expected business condition, not a bug); an unexpected deposit failure logs at ERROR via logger.exception() (Part 25.4), since it represents a genuine, unanticipated system problem worth full traceback capture.

31.3 Project: CSV Data Validation System

```
import csv

class RowValidationError(Exception):
    def __init__(self, row_number, errors):
        self.row_number = row_number
        self.errors = errors
        super().__init__(f'Row {row_number}: {"; ".join(errors)}')

def validate_row(row_number, row):
    errors = []
    if not row.get('email') or '@' not in row['email']:
        errors.append('invalid email')
    try:
        age = int(row.get('age', ''))
        if not (0 <= age <= 120):
            errors.append('age out of range')
    except ValueError:
        errors.append('age is not a number')
    if errors:
        raise RowValidationError(row_number, errors)

def validate_csv(path):
    valid_rows, invalid_rows = [], []
    try:
        with open(path, newline='', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row_number, row in enumerate(reader, start=2):
                try:
                    validate_row(row_number, row)
                    valid_rows.append(row)
                except RowValidationError as e:
                    invalid_rows.append(e)
    except FileNotFoundError:
        raise FileNotFoundError(f'CSV file not found: {path}') from None
```

```
return valid_rows, invalid_rows
```

Step by step: `validate_row()` collects every problem with a single row (not just the first one found) into a list before raising one combined `RowValidationError`, exactly mirroring the accumulate-then-raise pattern from Part 20.5. `validate_csv()` then wraps each row's validation independently so a single malformed row never aborts the rest of the file, and finally translates a missing file into a clearer `FileNotFoundError` message, following the file-handling patterns from Part 19.

31.4 Project: JSON Data Processing System

```
import json

class SchemaError(Exception):
    pass

REQUIRED_FIELDS = {'id': int, 'name': str, 'price': (int, float)}

def validate_schema(record, index):
    for field, expected_type in REQUIRED_FIELDS.items():
        if field not in record:
            raise SchemaError(f'Record {index}: missing field {field!r}')
        if not isinstance(record[field], expected_type):
            raise SchemaError(
                f'Record {index}: field {field!r} should be {expected_type}, got {type(record[field]).__name__}'
            )

def load_and_validate(path):
    try:
        with open(path) as f:
            data = json.load(f)
    except FileNotFoundError:
        raise FileNotFoundError(f'File not found: {path}') from None
    except json.JSONDecodeError as e:
        raise ValueError(f'Invalid JSON in {path} at line {e.lineno}: {e.msg}') from e

    if not isinstance(data, list):
        raise SchemaError('Expected a JSON array of records at the top level')
    valid, invalid = [], []
    for i, record in enumerate(data):
        try:
            validate_schema(record, i)
            valid.append(record)
        except SchemaError as e:
            invalid.append(str(e))
    return valid, invalid
```

This mirrors the CSV project's structure but for JSON: file-level failures (missing file, malformed JSON) are translated into clear exceptions immediately, while record-level schema failures are collected per-record without aborting the whole file, directly reusing the two-tier error handling strategy established in Part 19.7.

31.5 Project: API Data Extraction System

```
import requests, time

class ExtractionError(Exception):
    pass

def fetch_page(url, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            response = requests.get(url, timeout=10)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.HTTPError as e:
            if e.response.status_code == 429:
                wait = int(e.response.headers.get('Retry-After', 2 ** attempt))
                time.sleep(wait)
                continue
            raise ExtractionError(f'HTTP error {e.response.status_code} fetching {url}') from e
        except requests.exceptions.RequestException as e:
            if attempt == max_retries:
                raise ExtractionError(f'Failed to fetch {url} after {max_retries} attempts') from e
            time.sleep(2 ** attempt)

def extract_all_pages(base_url, max_pages=100):
    all_records = []
    for page in range(1, max_pages + 1):
        try:
            data = fetch_page(f'{base_url}?page={page}')
        except ExtractionError as e:
            logger.error(f'Stopping extraction at page {page}: {e}')
            break
        if not data.get('results'):
            break
        all_records.extend(data['results'])
    return all_records
```

Combines the API error-handling patterns from Part 22 with the retry and backoff logic from Part 30: rate limiting (429) is handled by waiting the server-specified duration and retrying the same attempt (using `continue` to avoid consuming a retry count), while other network failures use exponential backoff up to a maximum.

`extract_all_pages()` treats an extraction failure as a signal to stop gracefully and return whatever was already collected, rather than losing all previously fetched pages.

31.6 Project: Data Cleaning Pipeline

```
import pandas as pd

class PipelineStageError(Exception):
    def __init__(self, stage, original):
        self.stage, self.original = stage, original
```

```

        super().__init__(f"Stage '{stage}' failed: {original}")

def run_pipeline(df, stages):
    report = {'stage_results': [], 'final_rows': None}
    for name, fn in stages:
        try:
            before = len(df)
            df = fn(df)
            report['stage_results'].append(
                {'stage': name, 'rows_before': before, 'rows_after': len(df)}
            )
        except Exception as e:
            raise PipelineStageError(name, e) from e
    report['final_rows'] = len(df)
    return df, report

def drop_missing_critical(df):
    return df.dropna(subset=['customer_id', 'amount'])

def remove_negative_amounts(df):
    return df[df['amount'] >= 0]

def standardize_dates(df):
    df = df.copy()
    df['date'] = pd.to_datetime(df['date'], errors='coerce')
    return df.dropna(subset=['date'])

cleaned_df, report = run_pipeline(raw_df, [
    ('drop_missing_critical', drop_missing_critical),
    ('remove_negative_amounts', remove_negative_amounts),
    ('standardize_dates', standardize_dates),
])

```

Reuses the named-stage pipeline pattern from Part 21.4, adapted for tabular data cleaning: every stage's row count is tracked before and after, giving a built-in audit trail of exactly how much data each cleaning step removed, in addition to the exception handling that identifies precisely which stage fails if one does.

31.7 Project: Machine Learning Prediction Service

```

class PredictionServiceError(Exception):
    pass

class ModelNotLoadedError(PredictionServiceError):
    pass

class InvalidInputError(PredictionServiceError):
    pass

class PredictionService:
    def __init__(self, model_path, expected_features):
        self.model = None
        self.model_path = model_path

```

```

        self.expected_features = expected_features

    def load(self):
        try:
            with open(self.model_path, 'rb') as f:
                self.model = pickle.load(f)
        except FileNotFoundError:
            raise ModelNotLoadedError(f'Model file not found: {self.model_path}')
    from None
        except Exception as e:
            raise ModelNotLoadedError(f'Failed to load model: {e}') from e

    def predict(self, features: dict):
        if self.model is None:
            raise ModelNotLoadedError('Call load() before predict()')
        missing = set(self.expected_features) - set(features)
        if missing:
            raise InvalidInputError(f'Missing features: {sorted(missing)}')
        try:
            values = [features[f] for f in self.expected_features]
            return self.model.predict([values])[0]
        except (ValueError, TypeError) as e:
            raise InvalidInputError(f'Invalid feature values: {e}') from e

def handle_prediction_request(service, payload):
    try:
        result = service.predict(payload)
        return {'status': 200, 'prediction': result}
    except ModelNotLoadedError as e:
        logger.critical(f'Service misconfigured: {e}')
        return {'status': 503, 'error': 'Service temporarily unavailable'}
    except InvalidInputError as e:
        return {'status': 400, 'error': str(e)}
    except Exception:
        logger.exception('Unexpected prediction error')
        return {'status': 500, 'error': 'Internal server error'}

```

Combines nearly every major theme of this handbook into one realistic service: a domain-specific exception hierarchy (Part 10), model loading and inference exception handling (Part 21), and a centralized request handler (Part 29.8) that maps each specific exception type to an appropriate HTTP-style status code, with a CRITICAL-level log specifically for the service-misconfiguration case, since an unloaded model in production needs urgent operator attention rather than routine warning-level logging.

31.8 Project: AI Model Loading and Inference System With Config Validation

```

class InvalidConfigurationError(Exception):
    pass

def load_pipeline(config_path):
    try:
        with open(config_path) as f:
            config = json.load(f)

```

```

except (FileNotFoundError, json.JSONDecodeError) as e:
    raise InvalidConfigurationError(f'Cannot load config: {e}') from e
required = {'model_path', 'preprocessor_path', 'expected_features'}
missing = required - config.keys()
if missing:
    raise InvalidConfigurationError(f'Config missing keys: {sorted(missing)}')
service = PredictionService(config['model_path'], config['expected_features'])
try:
    service.load()
except ModelNotLoadedError as e:
    raise InvalidConfigurationError(f'Configured model could not be loaded:
{e}') from e
return service

```

A thin but important orchestration layer: it validates configuration structure before attempting to use any of it (fail-fast, Part 29.2), and translates a lower-level `ModelNotLoadedError` into the same `InvalidConfigurationError` family used for every other startup problem, so anything calling `load_pipeline()` has exactly one exception type to handle for any kind of startup failure.

31.9 Project: Production-Style Data Pipeline With Logging and Exception Handling

```

import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s: %(message)s')
logger = logging.getLogger('pipeline')

class PipelineError(Exception):
    pass

def run_daily_pipeline(source_path, output_path):
    logger.info(f'Starting pipeline run for {source_path}')
    try:
        df = pd.read_csv(source_path)
    except FileNotFoundError:
        logger.critical(f'Source file missing: {source_path}')
        raise PipelineError('Source file not found') from None

    try:
        df = drop_missing_critical(df)
        df = remove_negative_amounts(df)
        df = standardize_dates(df)
    except Exception as e:
        logger.exception('Data cleaning failed')
        raise PipelineError('Cleaning stage failed') from e

    if len(df) == 0:
        logger.warning('Pipeline produced zero valid rows; aborting write')
        raise PipelineError('No valid rows after cleaning')

    try:

```

```
        df.to_csv(output_path, index=False)
    except PermissionError as e:
        logger.critical(f'Cannot write output to {output_path}: permission denied')
        raise PipelineError('Output write failed') from e

    logger.info(f'Pipeline completed successfully: {len(df)} rows written to
{output_path}')
    return len(df)
```

This final project draws together nearly the entire handbook: module-level logging configuration (Part 25.2), a single custom `PipelineError` used consistently at every failure point (exception translation, Part 13.6 and Part 29.6), narrow try blocks around each distinct stage (Part 4.5), appropriate log severities ranging from INFO for routine progress to CRITICAL for conditions requiring urgent operator attention, and a final explicit business-rule check (zero valid rows) that is treated as a genuine pipeline failure even though no lower-level exception was raised at all, exactly the kind of deliberate, fail-fast validation described in Part 29.1.

32. Interview Questions

32.1 Beginner Questions

Q1. What is the difference between a syntax error and an exception?

Answer: A syntax error is detected before the program runs at all, while Python is parsing the code; it can never be caught with try/except, since the code never successfully compiles. An exception (a runtime error) occurs after execution has already begun, when a specific operation fails; it can be caught and handled with try/except. See Part 1.4-1.5.

Q2. What does the finally block guarantee?

Answer: finally always runs, regardless of whether the try block succeeded, raised a caught exception, raised an uncaught exception, or executed a return, break, or continue. It is the correct place for cleanup code that must happen no matter what. See Part 7.1.

Q3. What is the difference between except Exception and a bare except:?

Answer: except Exception: catches every exception that descends from Exception, but deliberately excludes KeyboardInterrupt, SystemExit, and GeneratorExit, since those inherit directly from BaseException instead. A bare except: catches absolutely everything, including those three, which can prevent a program from being cleanly stopped or exited. except Exception: is almost always the safer choice. See Part 5.7-5.8.

32.2 Intermediate Questions

Q4. What is the difference between raise and raise ... from ...?

Answer: A bare raise (with no arguments) inside an except block re-raises the exact currently-handled exception, unchanged. raise NewException(...) from e explicitly raises a different exception while declaring e as its direct cause, setting the new exception's __cause__ attribute and producing a traceback that clearly shows both exceptions and their relationship. See Parts 13 and 14.

Q5. Why should assert not be used to validate user input?

Answer: Assertions can be globally disabled by running Python with the -O optimization flag, which strips every assert statement from the program entirely. If input validation relies on assert, that validation could silently vanish in an optimized deployment. Ordinary input validation should always use a genuine raise statement instead. See Part 15.6-15.7.

Q6. Explain the difference between TypeError and ValueError with an example of each.

Answer: TypeError signals that an operand or argument is fundamentally the wrong kind of object, such as calling len(5) on an integer, which has no length concept at all. ValueError signals that an operand is the correct type but holds an inappropriate value, such as int('abc'), where a string was expected and received, but its content cannot be parsed as a number. See Part 3.4.

32.3 Advanced Questions

Q7. Explain `__cause__` vs `__context__`.

Answer: `__context__` is set automatically by Python whenever a new exception is raised while another exception is already being handled, regardless of the programmer's intent; it represents 'what else was happening'. `__cause__` is set only when `raise ... from ...` is used explicitly, and represents a deliberate, programmer-declared causal relationship. When `__cause__` is set, `__suppress_context__` is automatically set to `True`, so the default traceback printer shows the explicit chain and hides the (possibly incidental) implicit one. See Part 13.4.

Q8. Why does Python separate `BaseException` and `Exception` instead of having a single root class?

Answer: Because `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit` represent intentional control-flow signals (a user stopping the program, a deliberate exit call, a generator being closed), not application errors. If they inherited from `Exception`, a broad `except Exception:` handler, a very common and often necessary pattern, would risk accidentally catching and suppressing them, making it difficult or impossible to cleanly stop a program from inside code with broad exception handling. See Part 11.2-11.3.

Q9. What happens if a `return` statement exists in both a `try` block and its `finally` block?

Answer: The `finally` block's `return` silently overrides the `try` block's `return` (and would even override and discard a currently propagating exception, with no warning). This is considered a serious anti-pattern; `finally` should be reserved strictly for side effects like cleanup, never for control flow such as `return`, `break`, or `continue`. See Part 7.7.

32.4 Debugging Questions

Q10. Given this traceback, identify the root cause.

```
Traceback (most recent call last):
  File "app.py", line 12, in <module>
    total = calculate_total(cart)
  File "app.py", line 7, in calculate_total
    return sum(item['price'] * item['qty'] for item in cart)
  File "app.py", line 7, in <genexpr>
    return sum(item['price'] * item['qty'] for item in cart)
                                ~~~~^~~~~~
KeyError: 'qty'
```

Answer: At least one dictionary inside the `cart` list is missing the key `'qty'`. Reading bottom to top (Part 2.3): the exact failure is a `KeyError` for `'qty'`, occurring inside a generator expression on line 7, which is itself inside `calculate_total`, called from the top-level script. The fix is either to validate every cart item has the required keys before this calculation runs, or use `item.get('qty', 0)` if a missing quantity should default to zero rather than being treated as an error.

32.5 Code-Output Questions

Q11. What does this code print?

```
try:
    print('A')
    raise ValueError
    print('B')
except ValueError:
    print('C')
else:
    print('D')
finally:
    print('E')
```

Answer: Prints A, C, E, in that order. 'A' prints, then ValueError is raised, immediately skipping 'B'; the except clause matches and prints 'C'; else is skipped since an exception occurred; finally always runs and prints 'E'. See Part 8.5 for the complete flow table.

Q12. What does this code print?

```
def f():
    try:
        return 1
    finally:
        print('cleanup')

print(f())
```

Answer: Prints 'cleanup' first, then 1. Even though return 1 begins returning immediately, finally is guaranteed to run before the function actually hands control back to the caller. See Part 7.7.

32.6 Scenario-Based Questions

Q13. You are building a public library function that reads a config file. Should it catch FileNotFoundError internally or let it propagate? Justify your answer.

Answer: In most cases, it should let it propagate, or translate it into a clearer, library-specific exception (Part 13.6), rather than silently catching and hiding it. The function itself typically has no way to know whether a missing file should fall back to a default, prompt the user, or abort the whole program; that decision genuinely belongs to the calling code, which has more context. Silently swallowing it and returning an empty config, for instance, could mask a real deployment problem. See Part 16.3.

Q14. An API client function occasionally fails with a connection error. How would you decide whether to retry?

Answer: Distinguish retryable from non-retryable failures (Part 30.5): a ConnectionError or a 5xx server error is often transient and worth retrying, ideally with exponential backoff and a maximum retry count; a 4xx client error (bad request, unauthorized, not found) will fail identically on every retry and should not be retried, but should fail immediately and clearly instead.

32.7 System-Design-Style Questions

Q15. Design an exception-handling strategy for a web service that calls three internal microservices to fulfill one user request.

Answer: Key elements: (1) wrap each individual microservice call in its own narrow try/except so a failure in one is identifiable and does not get confused with the others (Part 4.5); (2) translate each microservice's own specific exceptions into a small set of consistent, service-level exception types (exception translation, Part 13.6/29.6); (3) decide per-dependency whether a failure should be fatal to the whole request or degrade gracefully with a fallback (Part 29.3), for example a non-critical recommendations service failing should not prevent the rest of the page from loading; (4) apply retry logic with backoff specifically to transient network failures, not to validation failures (Part 30); (5) have one centralized top-level handler mapping every anticipated exception type to an appropriate HTTP status, with a final broad safety net and full logging for anything unexpected (Part 29.8); (6) ensure no internal exception detail or traceback ever leaks into the external response (Part 27.8).

33. Practical Exercises

Exercises are organized by theme, spanning the major parts of this handbook, and grouped into Beginner, Intermediate, Advanced, and Debugging tiers. Attempt each one before reading its solution.

33.1 Beginner Exercises

Exercise B1

Write a function `safe_divide(a, b)` that returns the result of `a / b`, or the string `'Cannot divide by zero'` if `b` is zero, using `try/except`.

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return 'Cannot divide by zero'
```

Solution: Wraps only the single division operation in `try`, catches exactly `ZeroDivisionError` (not a bare `except`), and returns a clear fallback value. See Part 4 and Part 5.1.

Exercise B2

Write a function that asks for a dictionary key and prints its value, or `'Key not found'` if the key does not exist, without using `in` to check first.

```
def print_value(d, key):
    try:
        print(d[key])
    except KeyError:
        print('Key not found')
```

Solution: This is a small example of the EAFP style mentioned in Part 26.4: attempting the operation directly and catching the specific resulting exception, rather than checking with `in` first.

33.2 Intermediate Exercises

Exercise I1

Write a custom exception `NegativeValueError`. Write a function `square_root(x)` that raises it (with a clear message) if `x` is negative, and otherwise returns the square root using `x ** 0.5`.

```
class NegativeValueError(Exception):
    pass

def square_root(x):
    if x < 0:
        raise NegativeValueError(f'Cannot take the square root of a negative number: {x}')
    return x ** 0.5
```

Solution: Follows the custom exception pattern from Part 10.2: a small, focused exception class with a clear, specific message including the actual offending value.

Exercise I2

Write a function that reads a JSON file and returns its content, catching both `FileNotFoundError` and `json.JSONDecodeError`, printing a distinct, specific message for each.

```
import json

def load_json(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        print(f'{path} does not exist')
    except json.JSONDecodeError as e:
        print(f'{path} contains invalid JSON: {e}')
```

Solution: Two separate except clauses handle two genuinely different failure modes with distinct, specific messages, following the multiple-except pattern from Part 5.4 and the JSON handling from Part 19.6.

Exercise I3

Rewrite this overly broad try block to use narrow, specific exception handling:

```
try:
    data = fetch(url)
    parsed = json.loads(data)
    value = parsed['result']['value']
except Exception:
    value = None

try:
    data = fetch(url)
except requests.RequestException:
    value = None
else:
    try:
        parsed = json.loads(data)
        value = parsed['result']['value']
    except json.JSONDecodeError:
        value = None
    except KeyError:
        value = None
```

Solution: Splits the single giant try block into two purposeful stages, applying the Part 4.5 principle directly, and uses `else` (Part 6) to only attempt parsing once fetching has genuinely succeeded.

33.3 Advanced Exercises

Exercise A1

Implement a retry decorator that retries a function up to `max_retries` times with exponential backoff, only for a specified exception type.

```
import time, functools

def retry(exception_type, max_retries=3, base_delay=1):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_retries + 1):
                try:
                    return func(*args, **kwargs)
                except exception_type:
                    if attempt == max_retries:
                        raise
                    time.sleep(base_delay * (2 ** (attempt - 1)))
            return wrapper
        return decorator

@retry(ConnectionError, max_retries=3)
def fetch_data(url):
    return requests.get(url, timeout=5).json()
```

Solution: Generalizes the retry loop from Part 30.4 into a reusable decorator; the exception type to retry on is configurable, so the decorator is not hardwired to one specific failure mode, and the final attempt still re-raises rather than silently giving up, per Part 30.6.

Exercise A2

Design a small exception hierarchy for a file upload service that distinguishes: file too large, invalid file type, and virus scan failure. Write a function that raises the appropriate one and a handler that responds to each differently.

```
class UploadError(Exception):
    pass

class FileTooLargeError(UploadError):
    pass

class InvalidFileTypeError(UploadError):
    pass

class VirusDetectedError(UploadError):
    pass

def validate_upload(file):
    if file.size > MAX_SIZE:
        raise FileTooLargeError(f'File exceeds {MAX_SIZE} bytes')
    if file.extension not in ALLOWED_TYPES:
        raise InvalidFileTypeError(f'{file.extension} is not allowed')
```

```

if scan_for_virus(file):
    raise VirusDetectedError('File failed security scan')

def handle_upload(file):
    try:
        validate_upload(file)
    except FileTooLargeError as e:
        return {'status': 413, 'error': str(e)}
    except InvalidFileTypeError as e:
        return {'status': 415, 'error': str(e)}
    except VirusDetectedError as e:
        logger.critical(f'Virus detected in upload: {e}')
        return {'status': 422, 'error': 'File rejected'}

```

Solution: A complete small hierarchy (Part 10.4) mapped to distinct, meaningful HTTP-style status codes (Part 29.8); the security-relevant `VirusDetectedError` is logged at CRITICAL severity, since it may warrant investigation beyond simply rejecting the single upload.

33.4 Debugging Exercises

Exercise D1

This code is supposed to log and continue on error, but it crashes the whole program on the first bad record. Find and fix the bug.

```

for record in records:
    try:
        process(record)
    except ValueError:
        print(f'Skipping {record}')
    print(f'Finished {record}')
    raise

```

Solution: The stray `raise` at the end (outside the `except` block, at the loop's top level) unconditionally re-raises whatever exception is currently active in Python's internal state, or raises `RuntimeError: No active exception to re-raise if none is`, crashing the loop immediately after the first record regardless of success or failure. The fix is simply to delete that line entirely; it does not belong there at all. See Part 14.1 for the correct, intentional use of a bare `raise` strictly inside an `except` block.

Exercise D2

This function is meant to validate that age is a positive integer, but the check silently does nothing in production. Explain why and fix it.

```

def set_age(age):
    assert isinstance(age, int) and age > 0, 'Invalid age'
    self.age = age

```

Solution: `assert` statements are stripped out entirely when Python runs with the `-O` flag, so this validation can silently disappear in an optimized production deployment (Part 15.7). Since age likely originates from external input (a user, an API), this must be rewritten with an ordinary `raise`: `if not (isinstance(age, int) and age > 0): raise ValueError('Invalid age')`. See Part 15.5-15.6 for the `assert`-vs-validation distinction.

34. Complete Exception Handling Cheat Sheet

34.1 Common Exceptions Reference

Code Line	Explanation
ZeroDivisionError	Dividing by zero. Cause: unchecked denominator. Handling: validate divisor, or try/except.
IndexError	Sequence index out of range. Cause: off-by-one, empty collection assumption. Handling: check len() first, or try/except.
KeyError	Missing dictionary key. Cause: assumed key presence. Handling: dict.get(), 'in' check, or try/except.
TypeError	Wrong type for an operation. Cause: mismatched types, wrong argument count. Handling: validate/convert types at boundaries.
ValueError	Right type, invalid value/content. Cause: unparseable or out-of-range input. Handling: validate before conversion; try/except on conversion.
AttributeError	Attribute/method does not exist on object. Cause: wrong type assumed, None value, typo. Handling: check type or None first.
NameError / UnboundLocalError	Name not defined / local used before assignment. Cause: typo, scoping mistake. Handling: define before use; use global/nonlocal deliberately.
ImportError / ModuleNotFoundError	Module or name cannot be imported. Cause: typo, missing dependency, wrong environment. Handling: verify spelling and installation; catch for optional deps.
FileNotFoundError / PermissionError	Missing file / access denied. Cause: bad path, missing file, insufficient privileges. Handling: catch specifically; provide fallback or clear message.
RecursionError	Max recursion depth exceeded. Cause: missing/unreachable base case. Handling: verify base case; consider an iterative rewrite.
AssertionError	assert condition was False. Cause: violated internal invariant. Handling: fix the underlying bug; never use assert for external validation.
StopIteration	Iterator exhausted. Cause: manual next() past the end. Handling: usually handled automatically by for loops.

34.2 Syntax Templates

try/except

```
try:
    risky()
except SpecificError as e:
    handle(e)
```

try/except/else

```
try:
    risky()
except SpecificError as e:
    handle(e)
else:
```

```
only_on_success()
```

try/except/finally

```
try:
    risky()
except SpecificError as e:
    handle(e)
finally:
    always_runs()
```

try/except/else/finally

```
try:
    risky()
except SpecificError as e:
    handle(e)
else:
    only_on_success()
finally:
    always_runs()
```

raise

```
raise ValueError('message')
raise # bare re-raise, inside except only
raise NewError('msg') from e # explicit chaining
raise NewError('msg') from None # suppress chaining
```

Custom exceptions

```
class DomainError(Exception):
    pass

class SpecificError(DomainError):
    def __init__(self, detail):
        self.detail = detail
        super().__init__(f'Specific failure: {detail}')
```

Exception chaining

```
try:
    low_level_operation()
except LowLevelError as e:
    raise HighLevelError('translated message') from e
```

Assertions

```
assert condition
assert condition, 'message shown if condition is False'
```

Context manager for guaranteed cleanup

```
with open(path) as f:
    data = f.read()
# f.close() is guaranteed, even if an exception occurs above
```

Retry with exponential backoff

```
for attempt in range(1, max_retries + 1):
    try:
        return operation()
    except RetryableError:
        if attempt == max_retries:
            raise
        time.sleep(base_delay * (2 ** (attempt - 1)))
```

34.3 Final Decision Checklist

Code Line	Explanation
Should I catch this?	Only if you have a specific, meaningful response; otherwise let it propagate.
How specific should the except be?	As specific as possible; avoid bare except: always, avoid except Exception: except at genuine top-level boundaries.
Should I re-raise?	Yes, if this handler only logs/alerts/cleans up but cannot resolve the problem.
Should I translate the exception?	Yes, if the caller should not need to know about a low-level implementation detail; always chain with from e.
Should I create a custom exception?	Yes, if the failure has real domain meaning, needs structured data, or callers benefit from catching it specifically.
Should I log it?	Yes, at an appropriate severity, using logging (not print), with logger.exception() inside except blocks.
Should I retry it?	Only if the failure is genuinely transient; never retry validation/client errors; always bound retries.
Am I leaking sensitive data?	Check every log message and user-facing error for credentials, tokens, or personal data before shipping.

Closing Note

Exception handling is not a peripheral topic bolted onto 'real' programming; it is a core discipline that determines whether software is merely functional in the easy cases or genuinely robust in the messy, unpredictable conditions of real use, real data, real networks, and real users. The difference between a fragile script and production-grade software is rarely the happy path; it is almost always how carefully and deliberately the failure paths were designed.

This handbook has moved from the absolute fundamentals, what an error even is, through the complete mechanics of try/except/else/finally, custom exceptions and the full exception hierarchy, and into the professional practices, chaining, logging, retries, design patterns, that separate hobbyist code from software that can be trusted to run unattended in production. Returning to Part 27's best-practices checklist and Part 34's cheat sheet as ongoing references, while keeping the underlying reasoning from each part in mind, is the surest path to writing Python that fails, when it must fail, clearly, safely, and informatively.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer